IEEE VIS 2026 Tutorial

Introduction to Quantum Computing for Visualization Researchers

Organizers

Hyeok Kim
KAIST

Leilani Battle
University of Washington

Sara Mouradian
University of Washington

Preparation material

https://hqci-tutorial.github.io/

If you haven't done the preparation steps,
please visit this website and complete it before the session.

It will take about 10–15 minutes.

Agenda

Opening

Introduction to quantum computing
An overview of fundamental concepts and algorithms

Interacting with quantum computer
Entanglement, simple search circuit

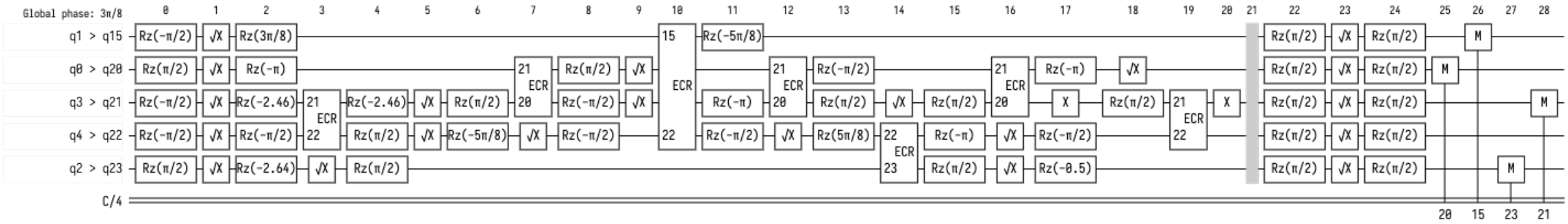
Recent Vis/HCI research in quantum computing
Analytic interfaces, programming tools, visualizations, and education

Brainstorming future research
Group discussion

Why do we organize this tutorial?

Quantum computing involves lots of data,
but we still lack good solutions for how to understand such data.

Program structure



What are their meanings?

What do we debug in this program?

How will the output be processed?

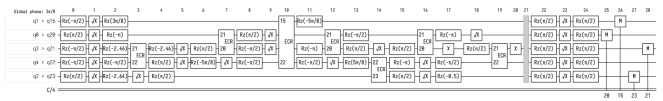
Why do we organize this tutorial?

Quantum computing involves lots of data,
but we still lack good solutions for how to understand such data.

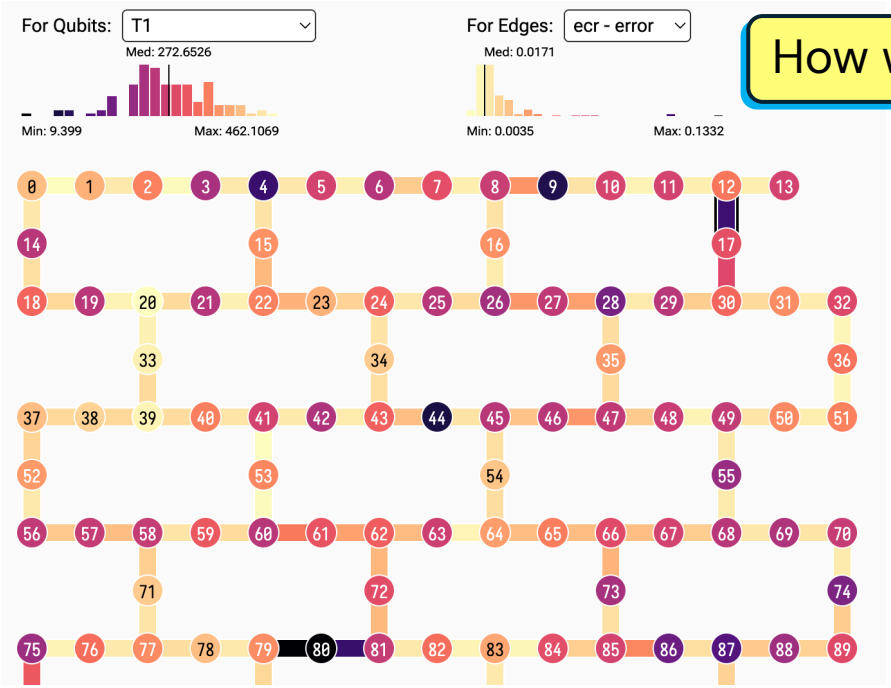
Is this method scalable?

How will this information affect my code?

Program structure



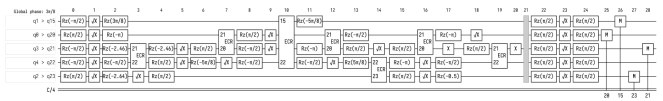
Machine assessment



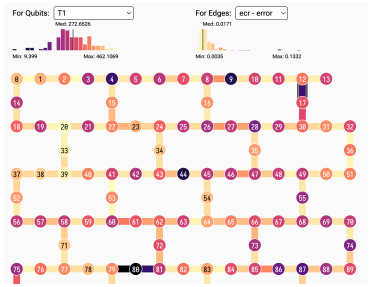
Why do we organize this tutorial?

Quantum computing involves lots of data,
but we still lack good solutions for how to understand such data.

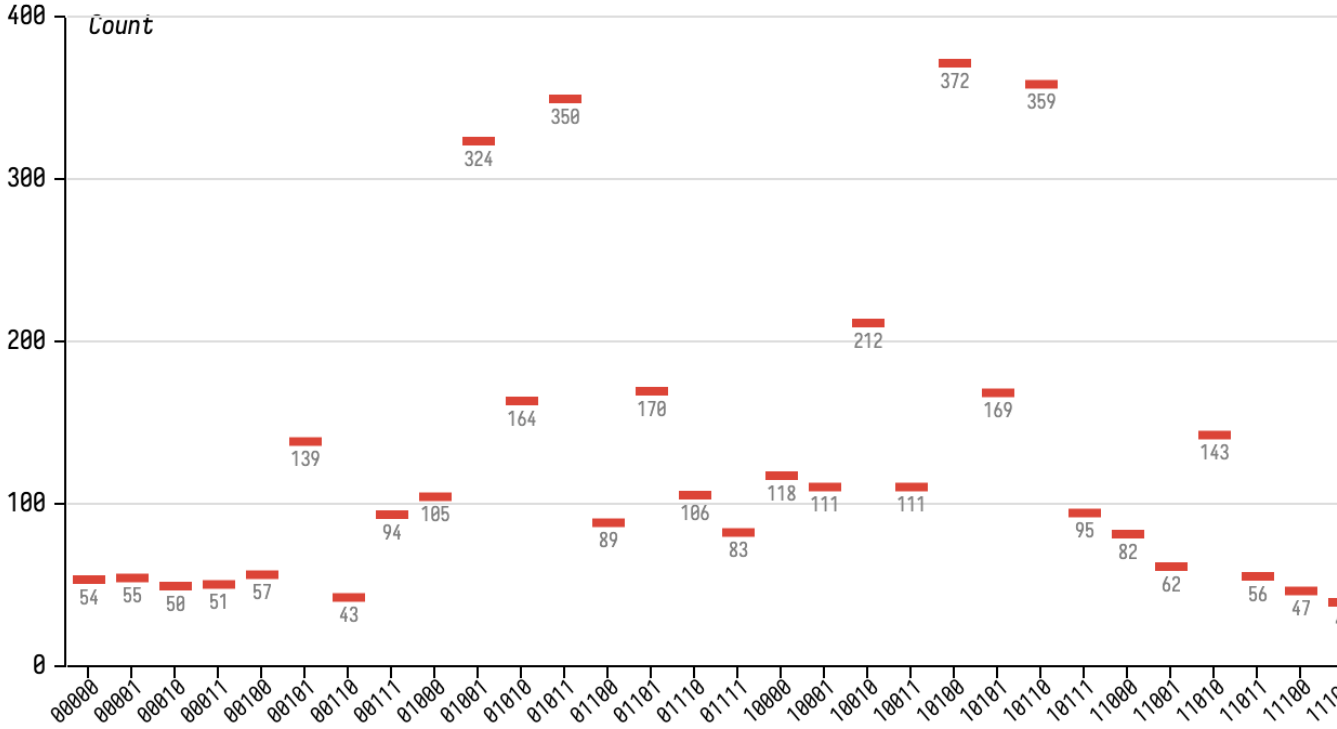
Program structure



Machine assessment



Measured outcomes



Is this outcome credible?

How to interpret this 01010101?

Objectives

Demystification

We will

Introduce core concepts in quantum computing

You will

Be aware of common vocabulary in quantum computing and feel more ready to make collaborations

Engagement

We will

Help you interact with a quantum computer

You will

Have a clear picture of how quantum computers work

Brainstorming

We will

Provide a discussion session for potential future research

You will

Be able to draw future visualization research with respect to quantum computing

Preparation Check

1. Have you created your IBM account (IBMid)? Can you sign in IBM Quantum Platform?
   (If you have trouble in credit card verification, it is fine to skip this)

2. Do you have the latest Python 3?

3. Have you installed Jupyter Lab (preferred)/Notebook?

4. Have you installed required packages?

5. Can you successfully run our test code?

Disclaimer

There are multiple methods/paradigms
for enabling, operating, and using quantum computers.

Our goal is to introduce one of them:
digital, superconductor-based quantum computing.

Quantum computing is a fast-evolving area.
What is in our materials may change drastically in the future.

Slides and codes will be distributed after this session.

There are some math. If you don't understand them right away,
it is mostly fine. You can revisit our materials and refer to our materials.

Part 1

Introducing Quantum Computing

Topics covered

Quantum computers—qubit, gate, measure

Superposition, Bloch sphere, and entanglement

Shor's algorithm

Search algorithm (oracle-based)

Optimization problems

Quantum error correction (and mitigation)

How I interpret this whole process

A quantum program = A giant matrix multiplication

For application scale,
a few thousand time multiplications of 100s-by-100s matrices

Composing a quantum program
= modeling a Bayesian probability distribution

Running a quantum program
= getting a posterior predictive distribution

Quantum computing is useful
for problems that need to test lots of candidates

What is a quantum computer?

A computing device that utilizes quantum mechanical properties:
superposition, **entanglement**, and **interference** of quantum particles. We will cover these concepts.

- IBM's superconductor machine
- Xanadu's photonic machine (Aurora)
- IonQ's trapped ion chip
- Neutral atoms
- Neutral molecules

What is a quantum computer?

Quantum computing provides promising solutions
for classically intractable problems (e.g., search, optimization, etc.)

Quantum processing unit (QPU)

A QPU is a chip that processes quantum operations.

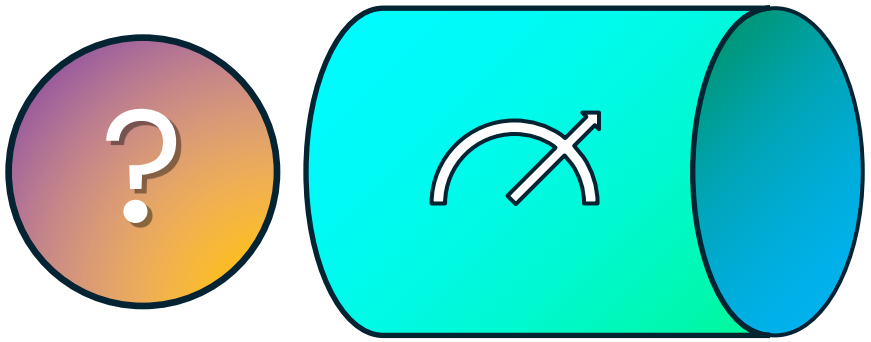
The primary component of a QPU is **qubits**.

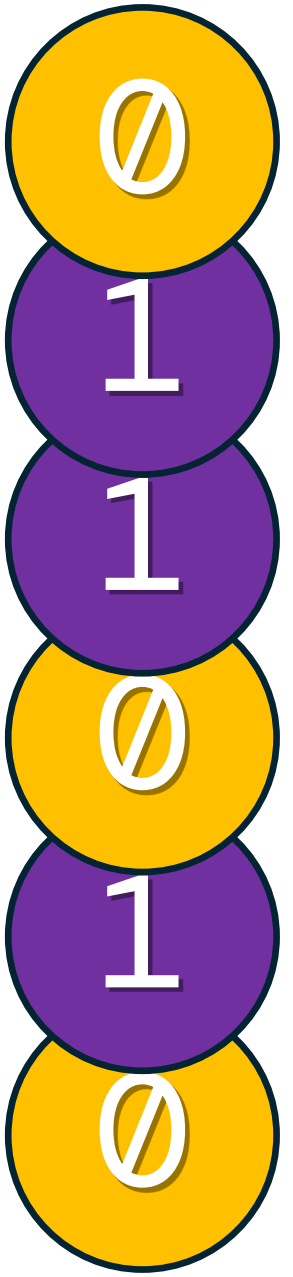
Qubit (=quantum bit)

A qubit exhibits superposition, entanglement, and interference.

Unlike classical bits with discrete 0 or 1 states,
qubits have superposed states

A qubit maintains a certain superposed state,
and is measured to be 0 or 1 accordingly
(not that it can be 0 or 1 at the same time).





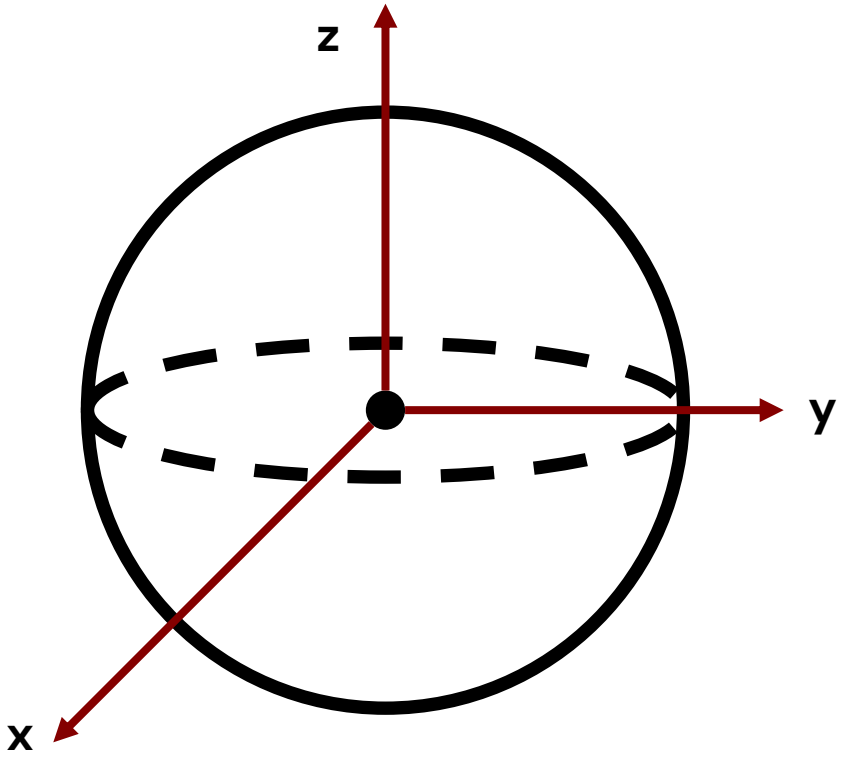
Superposition
Qubit state existing in a probabilistic distribution of a discrete outcome space

State
A mathematical way to represent quantum elements.

We manipulate superposition, entanglement and interference in a desired way.

Bloch sphere

A Bloch sphere visually represents a qubit's state by projecting it on a 3D surface.



It looks like 3D, but it is a 2D space mathematically.

Interactive visualization: https://bits-and-electrons.github.io/bloch-sphere-simulator

Mathematical look

A qubit's state can be formally expressed as:

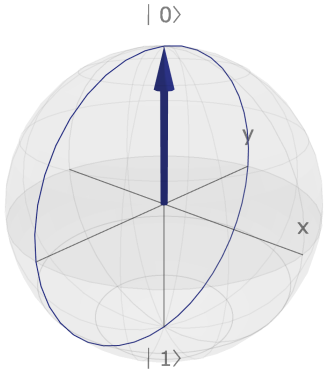
Ket Psi Alpha ket zero Beta ket one

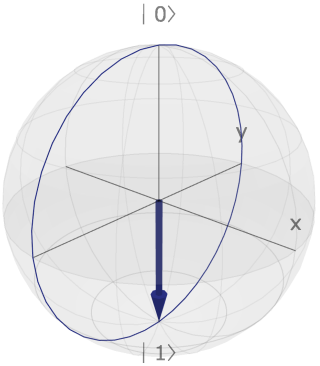
$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Ket?

A ket is a notation for a column vector.

In quantum computing, typically,





$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Mathematical look

Ket Psi Alpha ket zero Beta ket one

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

$$= \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

To simply put, it is a coordinate on the surface of a Bloch sphere, given an axis (in this case, the *Z* axis).

Mathematical look

Ket Psi Alpha ket zero Beta ket one

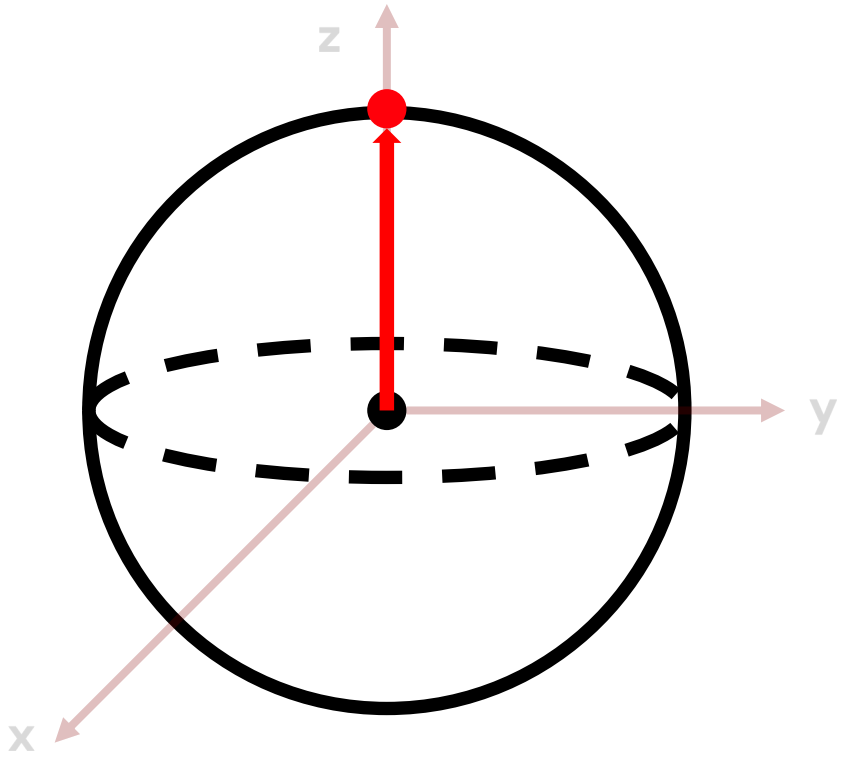
$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

$$|\alpha|^2 + |\beta|^2 = 1$$ *Born Rule*

$$P(\psi=0) = |\alpha|^2 \qquad P(\psi=1) = |\beta|^2$$

This is **superposition**: a qubit's state existing in a probabilistic distribution.
We want to manipulate qubits to have a certain superposed state to obtain desired outcomes.

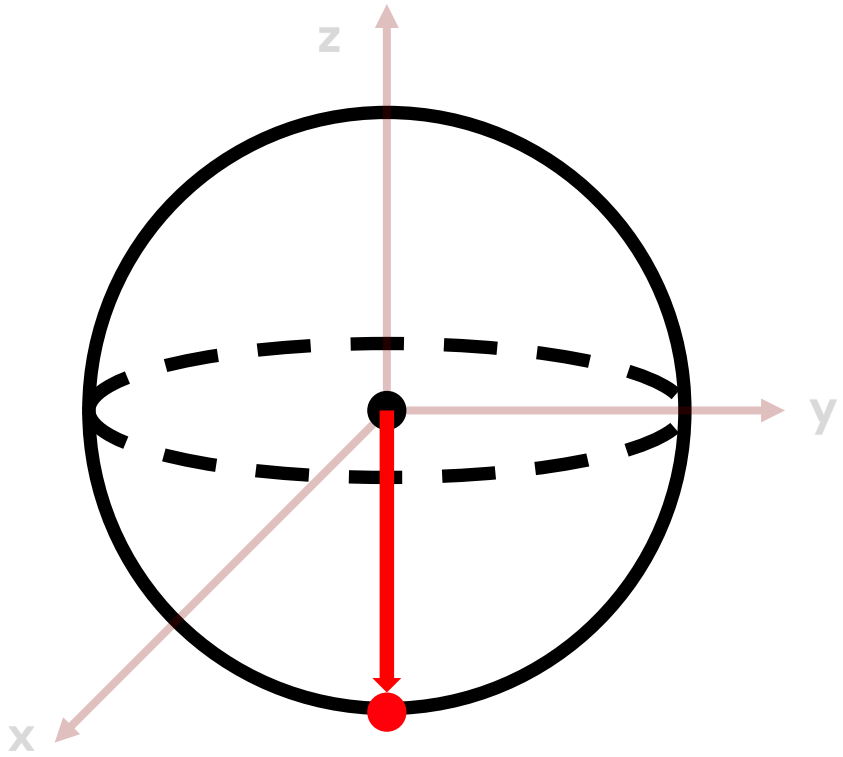
Common states in quantum computing



$$|0\rangle = 1|0\rangle + 0|1\rangle$$

$$P(0) = 1 \qquad P(1) = 0$$

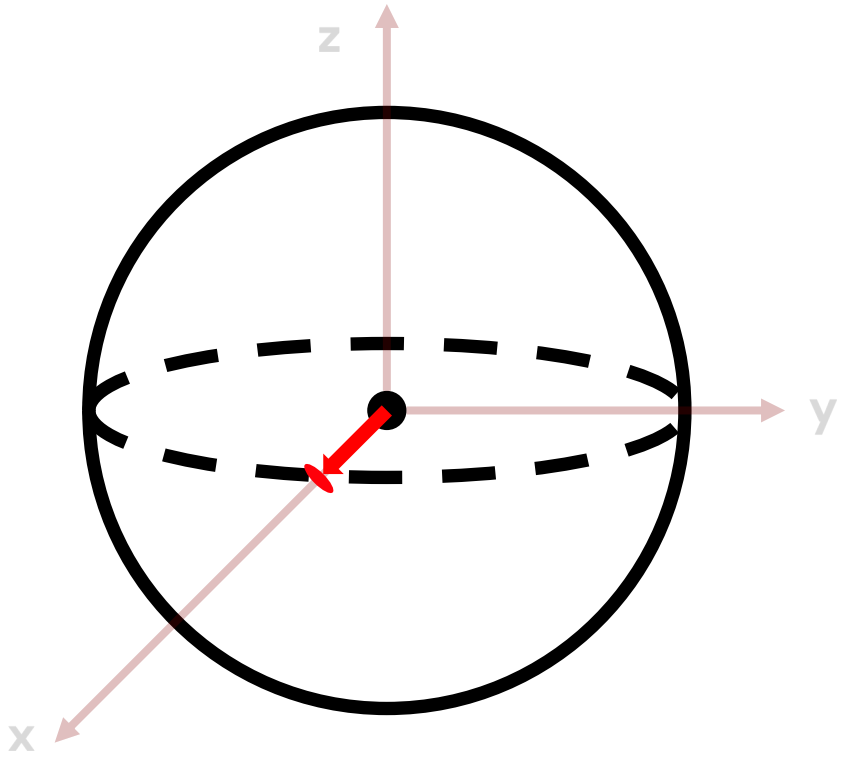
Common states in quantum computing



$$|1\rangle = 0|0\rangle + 1|1\rangle$$

$$P(0) = 0 \qquad P(1) = 1$$

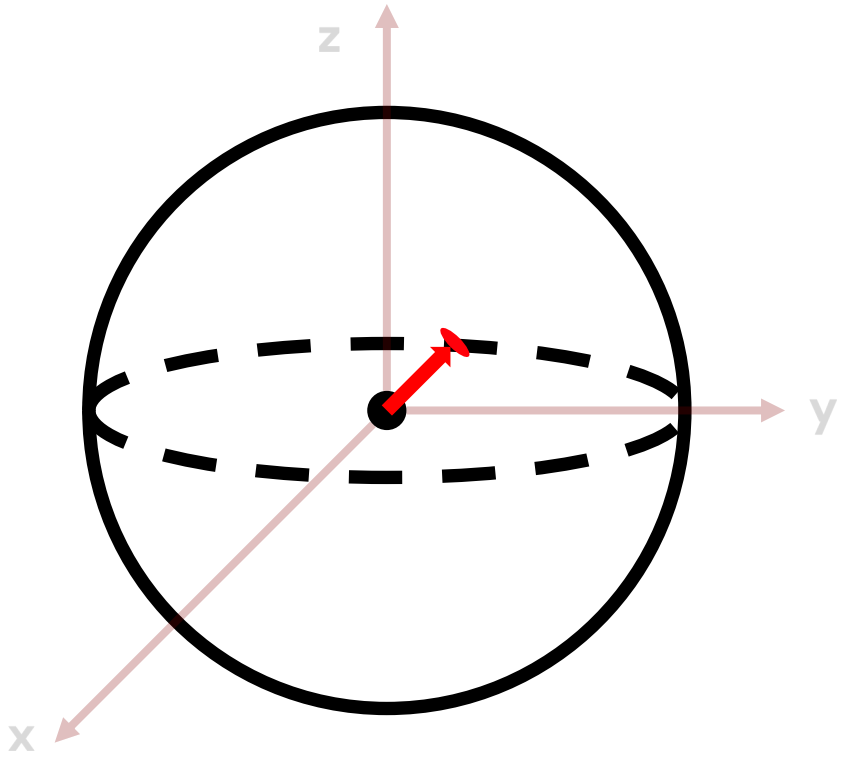
Common states in quantum computing



$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

$$P(0) = .5 \quad P(1) = .5$$

Common states in quantum computing

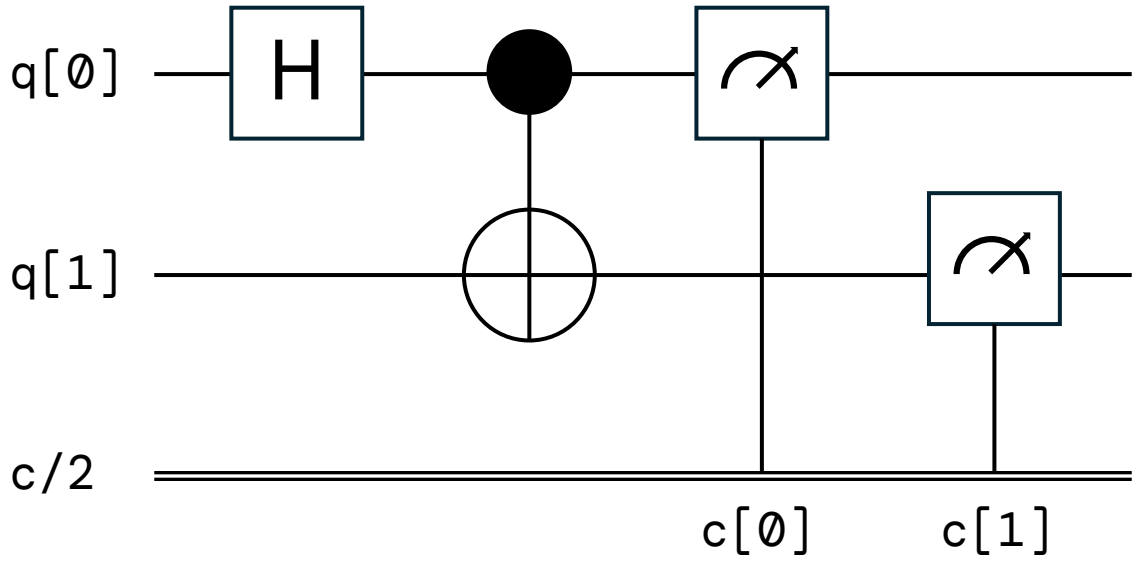


$$|-\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

$$P(0) = .5 \quad P(1) = .5$$

Quantum program diagram

A very traditional diagram for representing quantum programs.

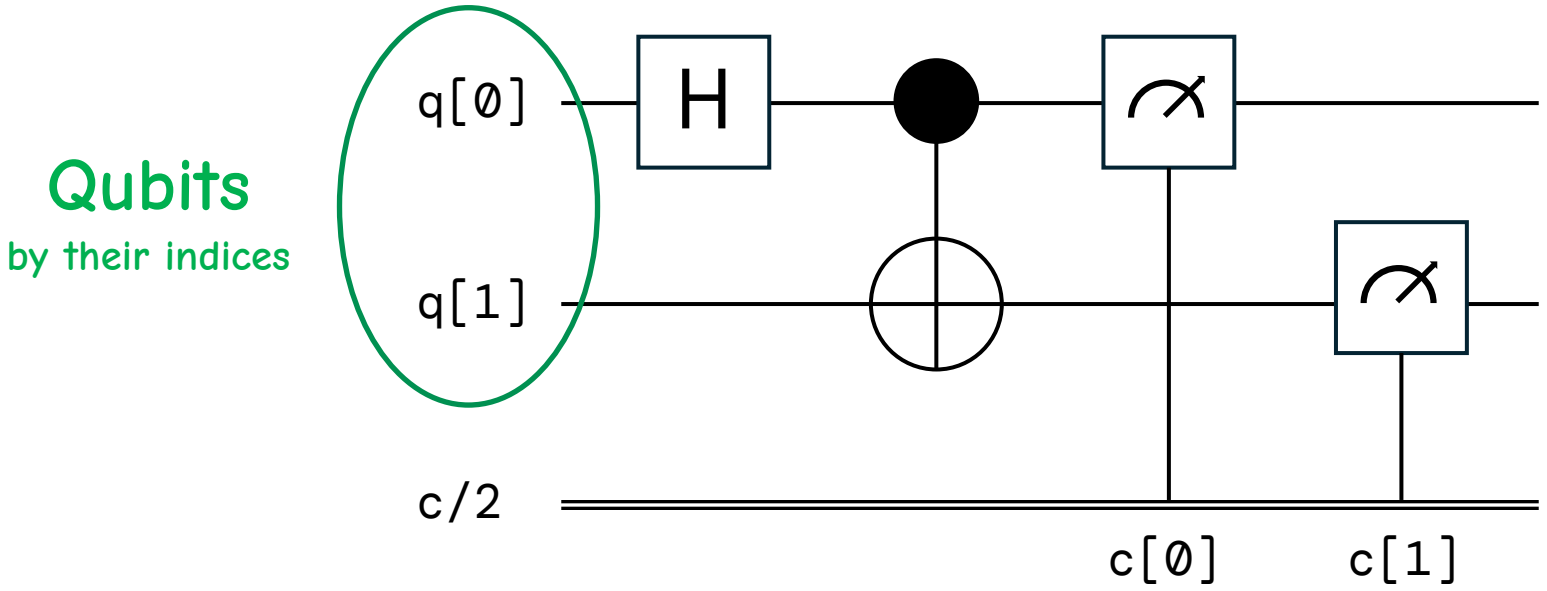


Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Quantum program diagram

A very traditional diagram for representing quantum programs.

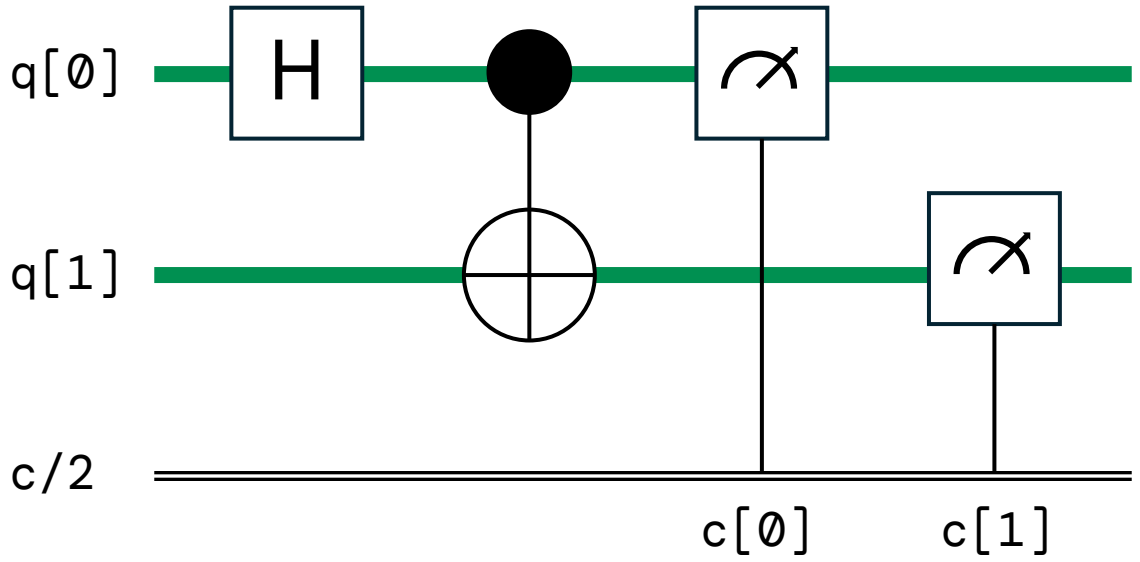


Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Quantum program diagram

A very traditional diagram for representing quantum programs.



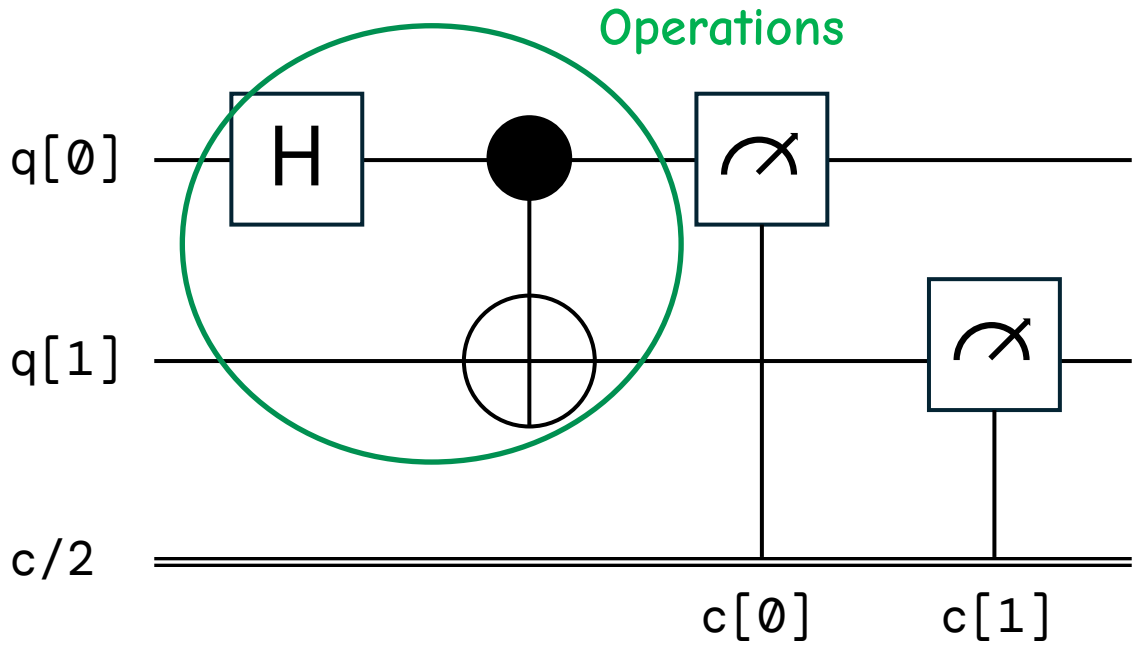
Qubit lines
(indicating to which qubit operations are applied)

Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Quantum program diagram

A very traditional diagram for representing quantum programs.

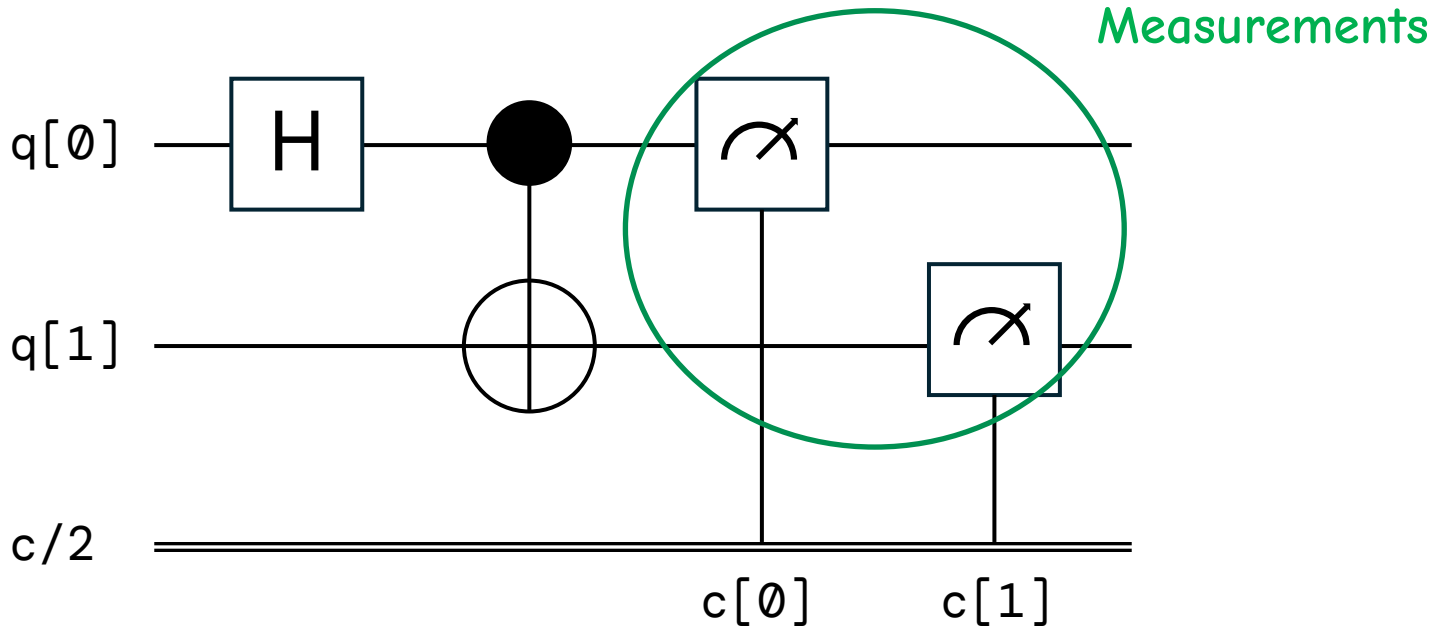


Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Quantum program diagram

A very traditional diagram for representing quantum programs.

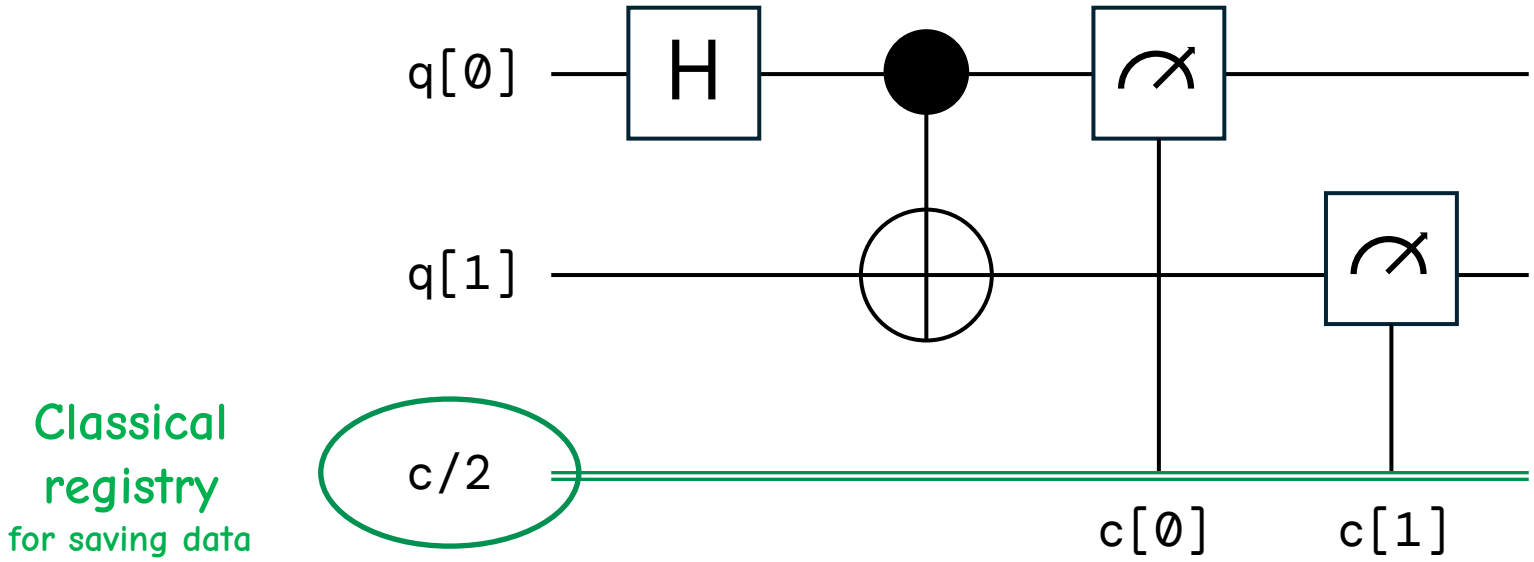


Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Quantum program diagram

A very traditional diagram for representing quantum programs.

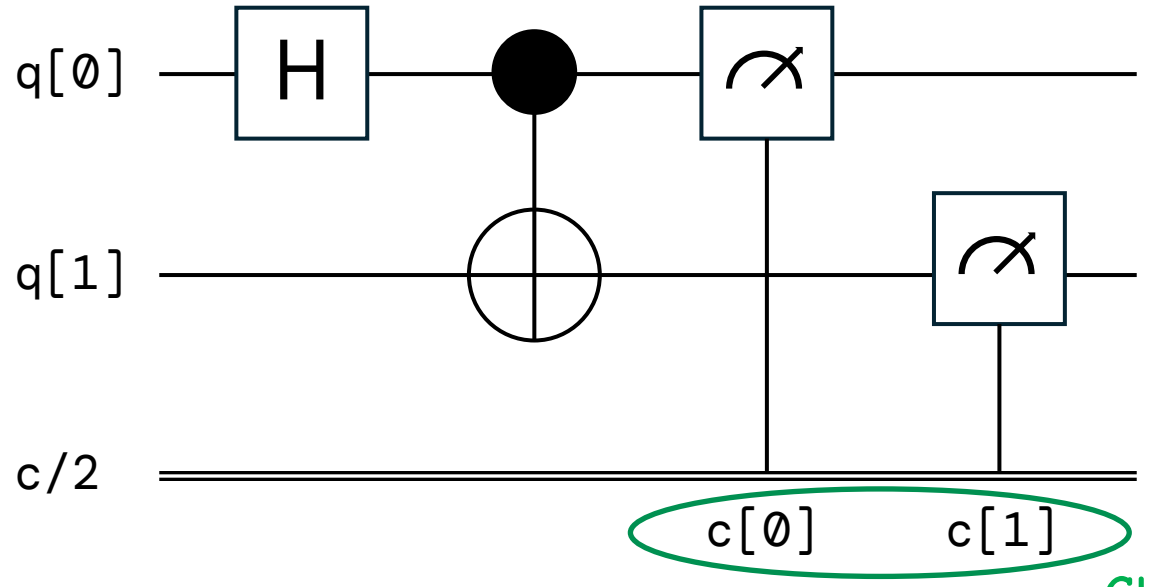


Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Quantum program diagram

A very traditional diagram for representing quantum programs.

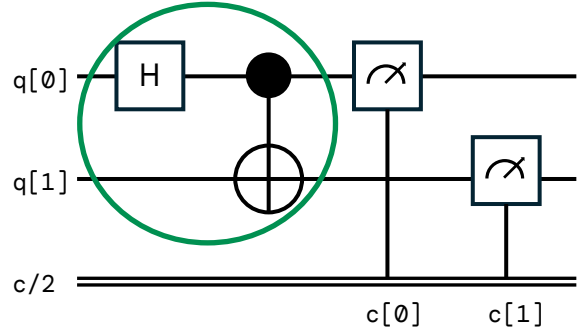


Classical bits
that store the measured outcomes
by their indices

Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Gate (or operation)



A gate manipulates a qubit's state (i.e., changing the alpha and beta values).

A gate must be expressed as a unitary matrix*.
(Its inverse = its conjugate transpose)

Original

$$M = \begin{pmatrix} a_0 + b_0 i & a_1 + b_1 i \\ a_2 + b_2 i & a_3 + b_3 i \end{pmatrix}$$

Inverse

$$M^{-1} = Det(M) \begin{pmatrix} a_3 + b_3 i & -a_1 - b_1 i \\ -a_2 - b_2 i & a_0 + b_0 i \end{pmatrix}$$

Conjugate transpose

$$M^{\dagger} = \begin{pmatrix} a_0 - b_0 i & a_2 - b_2 i \\ a_1 - b_1 i & a_3 - b_3 i \end{pmatrix}$$

Unitary

$$M^{-1} = M^{\dagger}$$

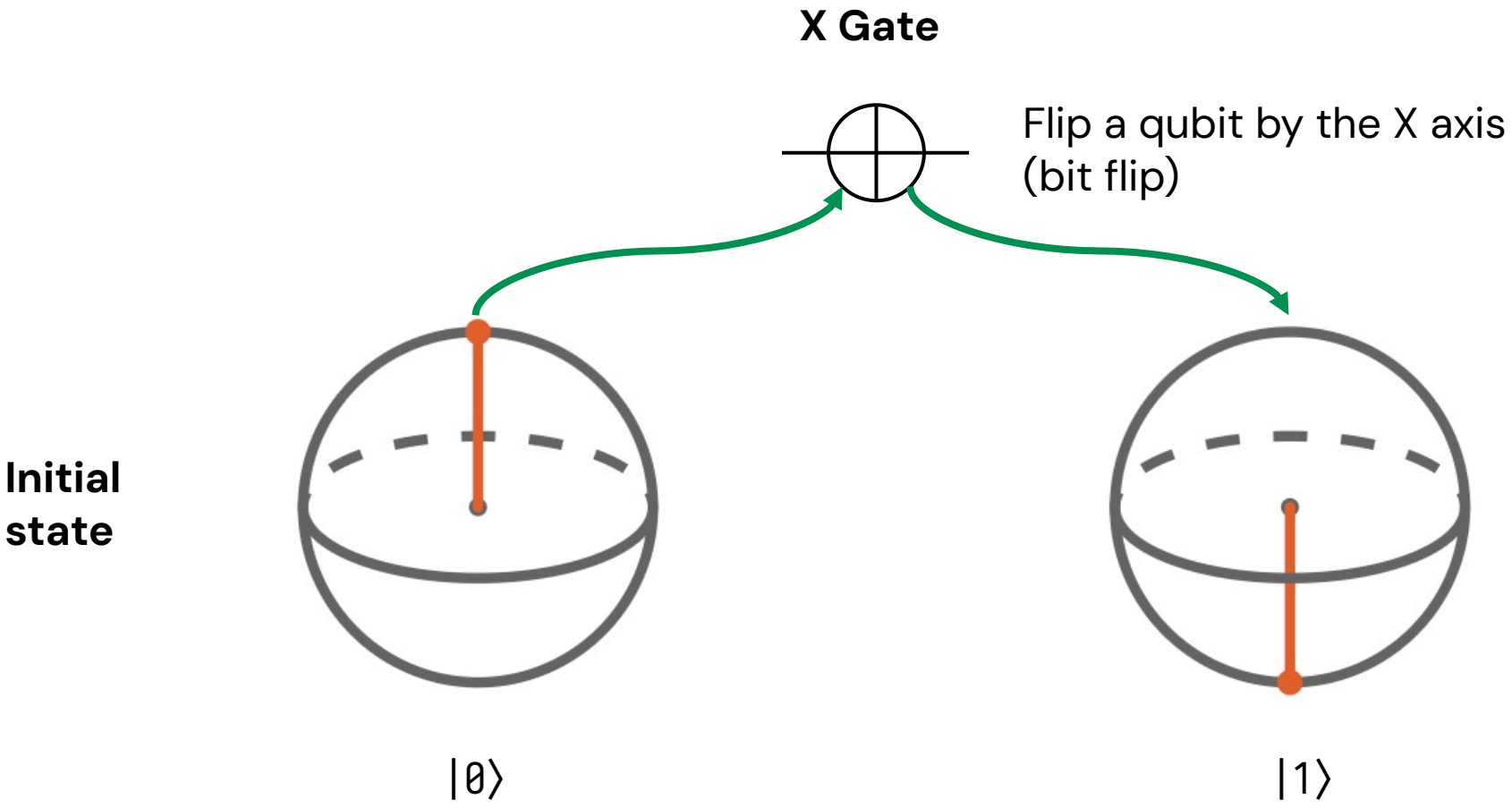
Transpose by the main axis
and change the imaginary number signs.

Example with a Y gate

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Y^{-1} = \frac{1}{0 - i(-i)} \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}, Y^{\dagger} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

$$YY^{\dagger} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} -i \cdot i & 0i - 0i \\ 0i - 0i & i \cdot (-i) \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Common gates and notations (single)

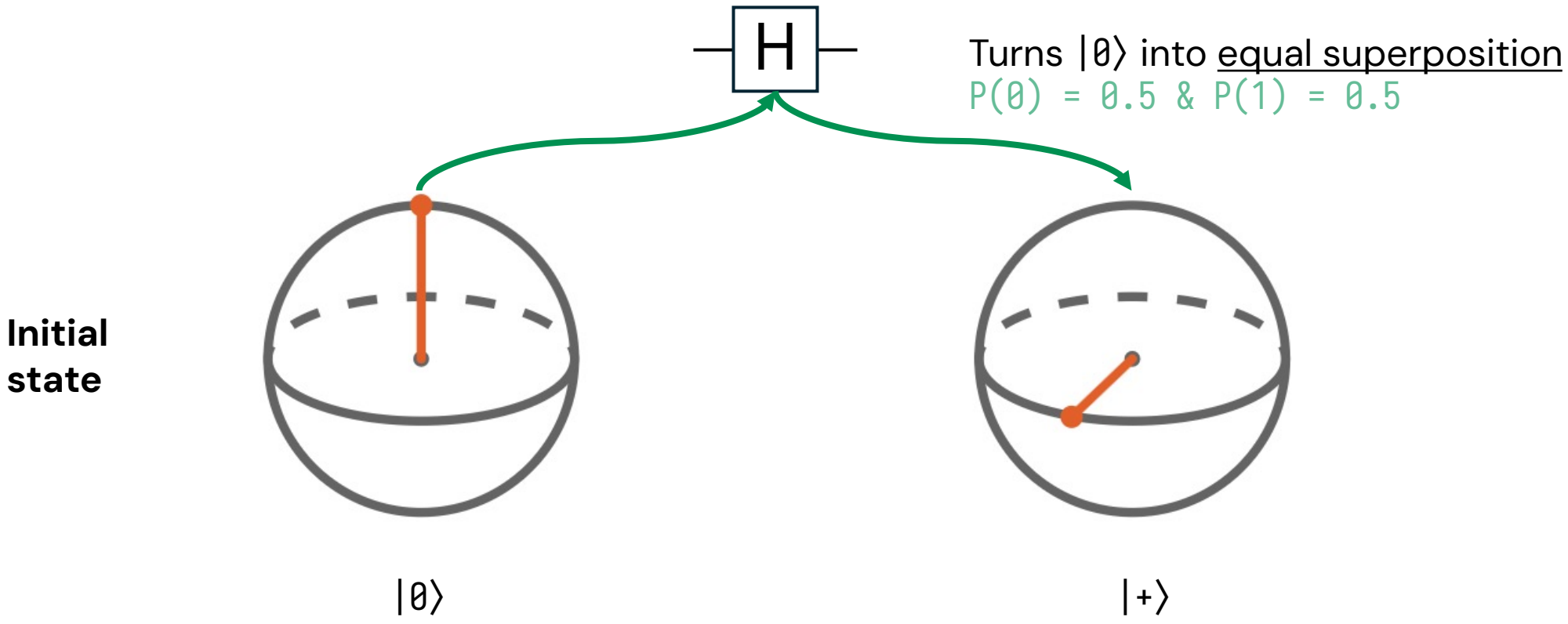


Common gates and notations (single)

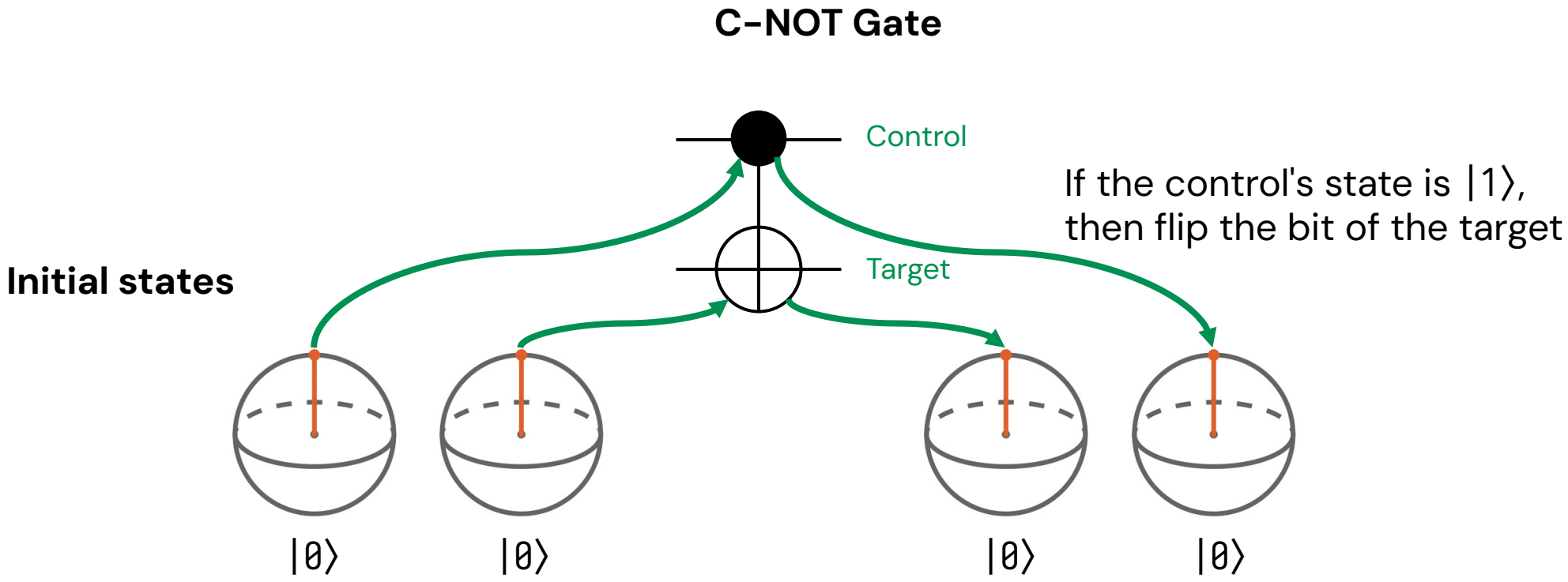
H Gate

Rotate by π along X and Z axis
(or flip against the Y axis)

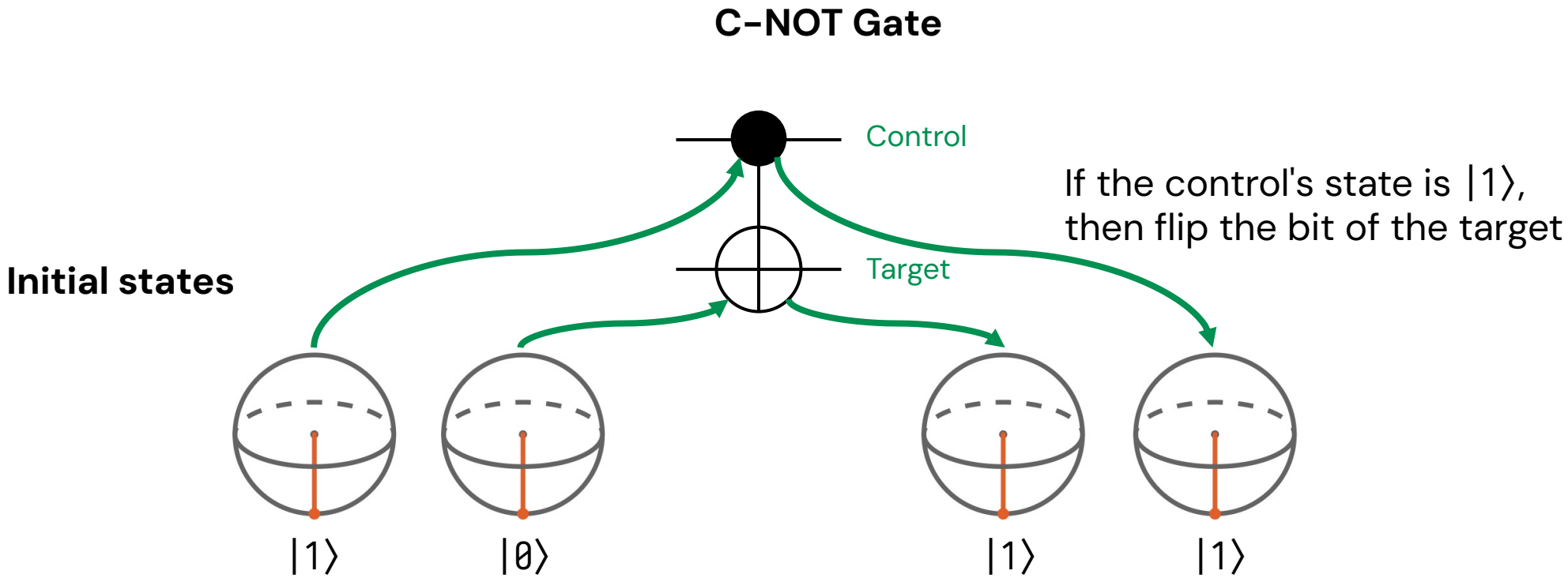
Turns |0⟩ into equal superposition
P(0) = 0.5 & P(1) = 0.5



Common gates and notations (multi)



Common gates and notations (multi)

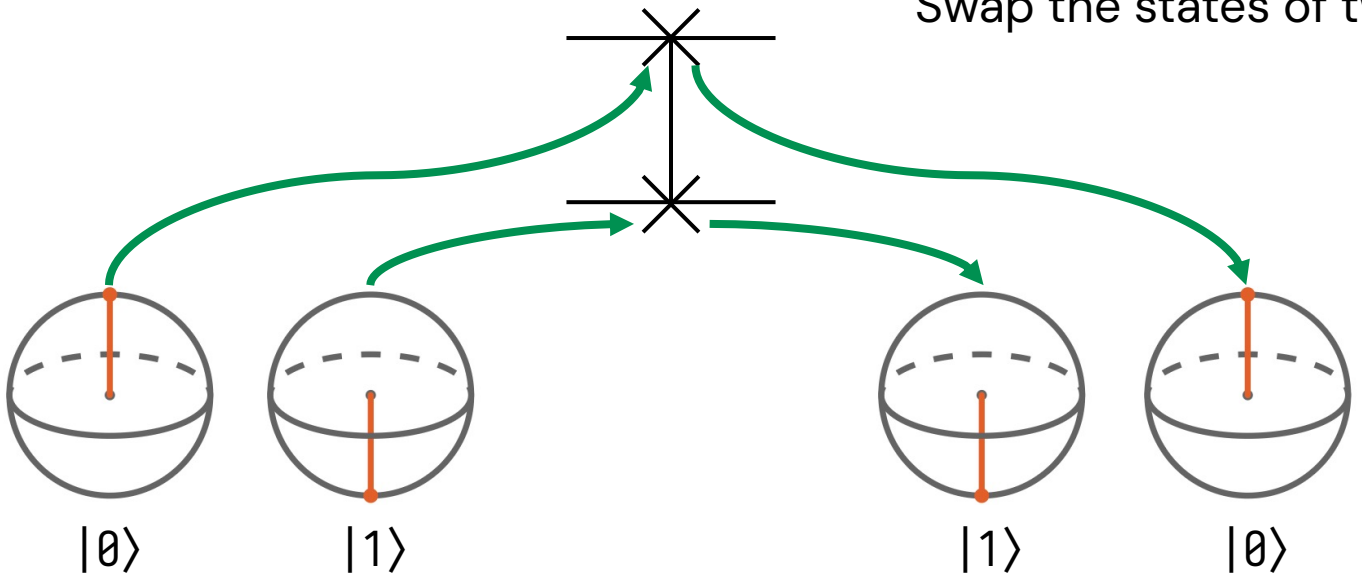


Common gates and notations (multi)

Swap Gate

Swap the states of two qubits

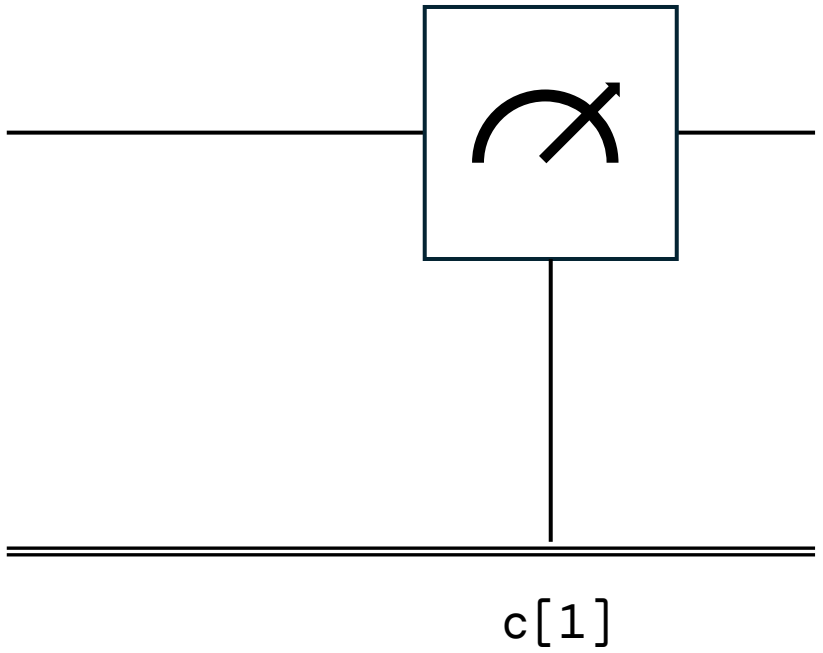
Initial states



Measure

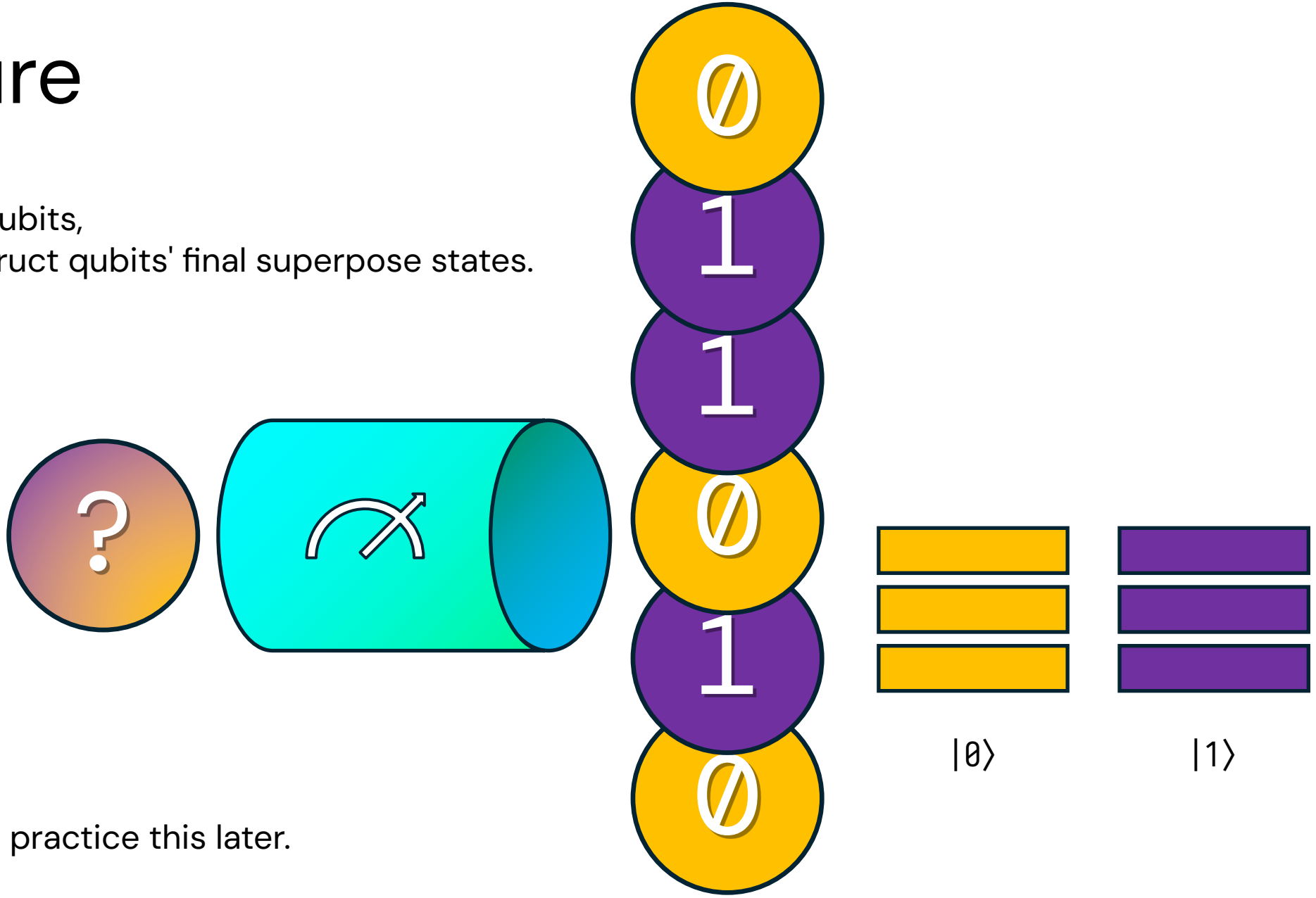
A most important property of quantum particles:
when they are measured (=interacted with external energy),
its superposed quantum state collapses (decoherence).

We can only measure a qubit through a certain axis.
We commonly use the Z axis (computational axis).



Measure

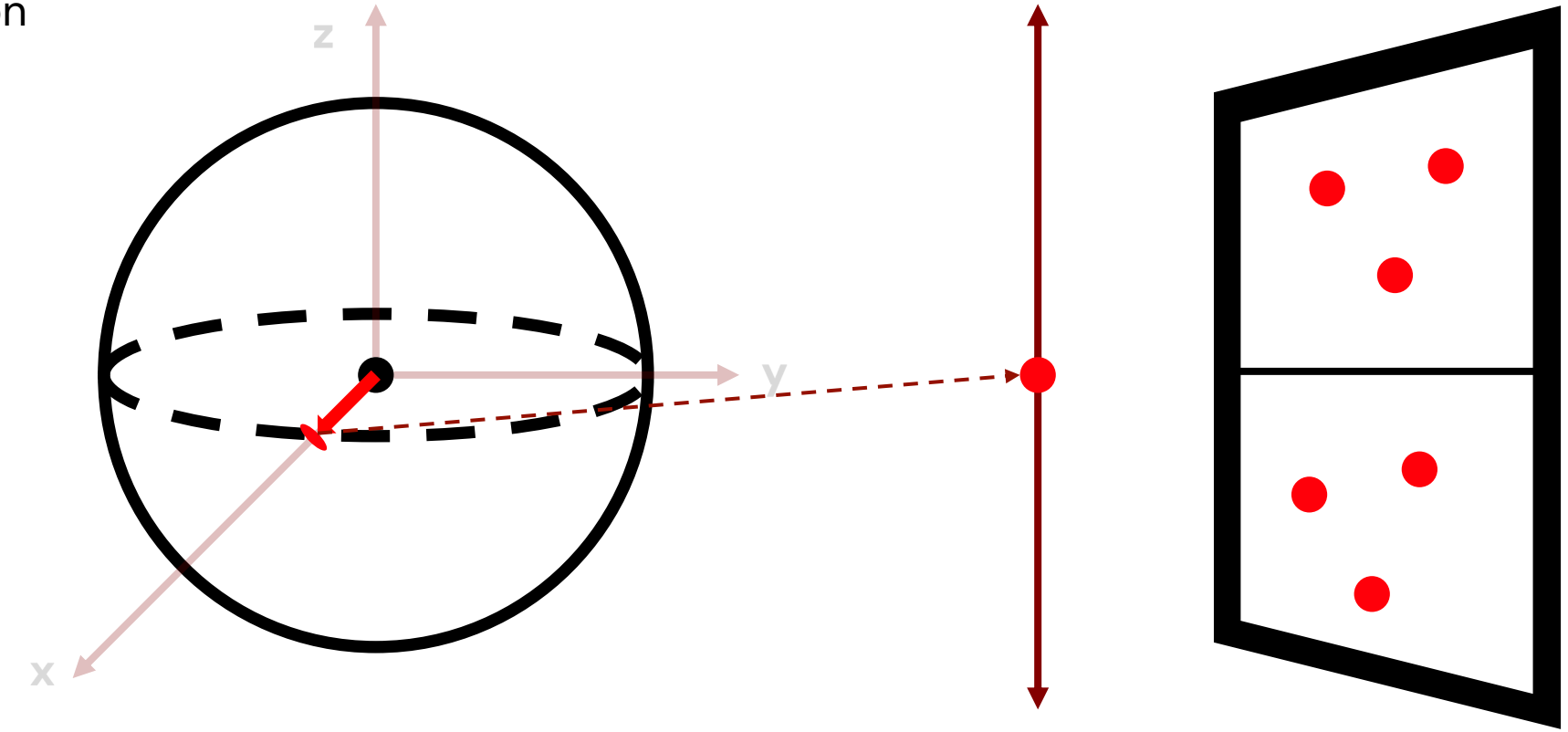
By measuring qubits,
we can reconstruct qubits' final superpose states.



We are going to practice this later.

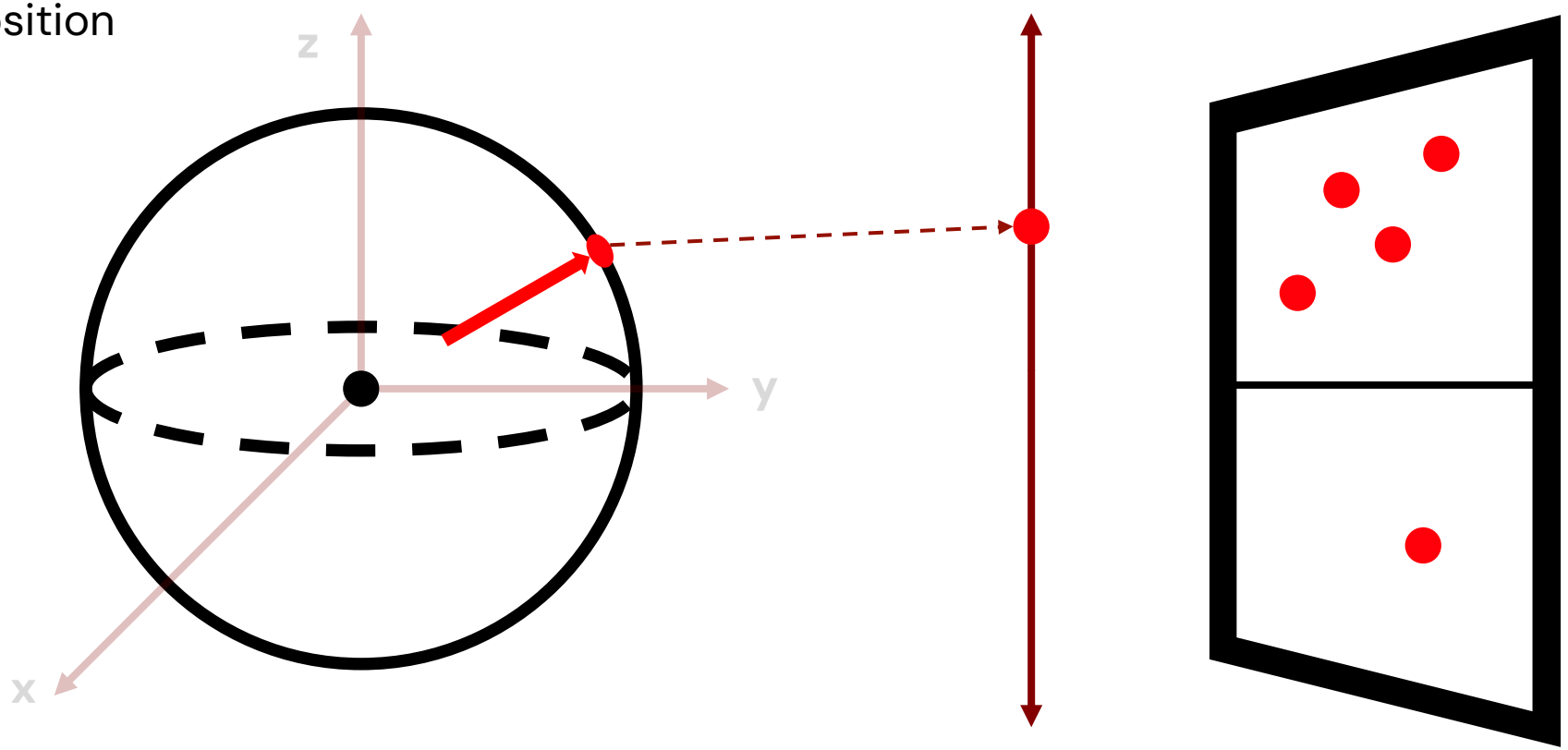
Measure along the Z axis

Equal superposition



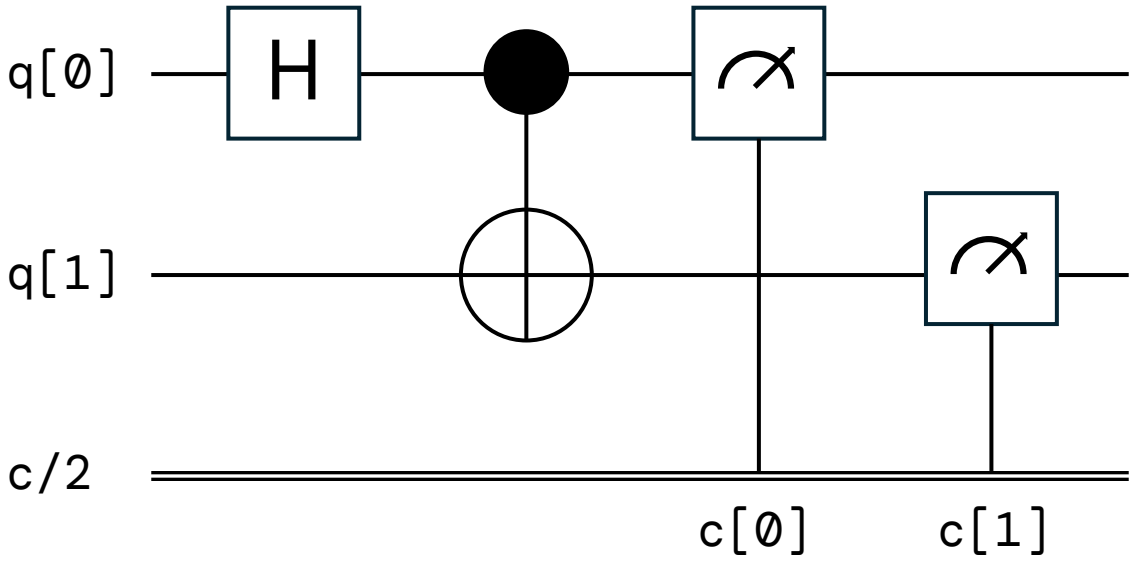
Measure along the Z axis

Non-equal superposition



Repeating circuits

Repeat this process MANY times (a few thousands at minimum).
Each run takes less than a millisecond (typically).
We'll get back to why later.

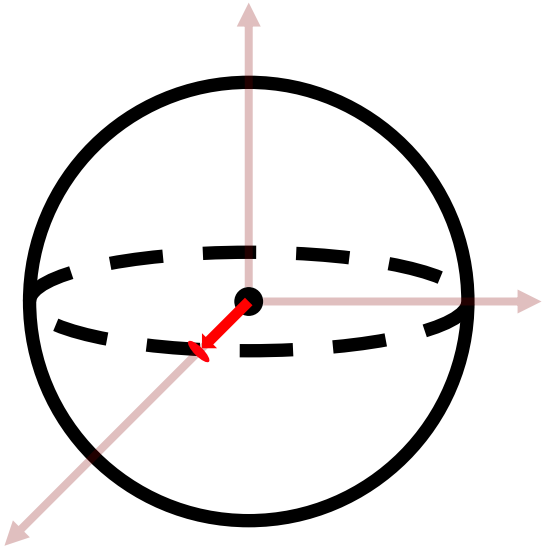




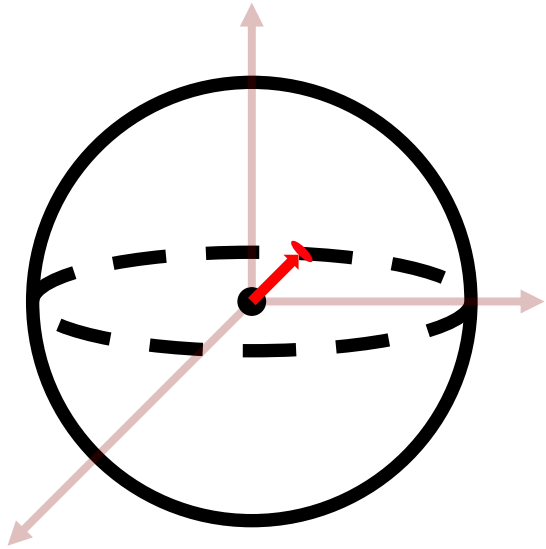
Terminologies will be explained.

Specifics may differ by toolkits, paradigms, and purposes.
Check out our paper on a quantum circuit visualization design space.

Common states in quantum computing



$$P(0) = \frac{1}{2} \qquad P(1) = \frac{1}{2}$$

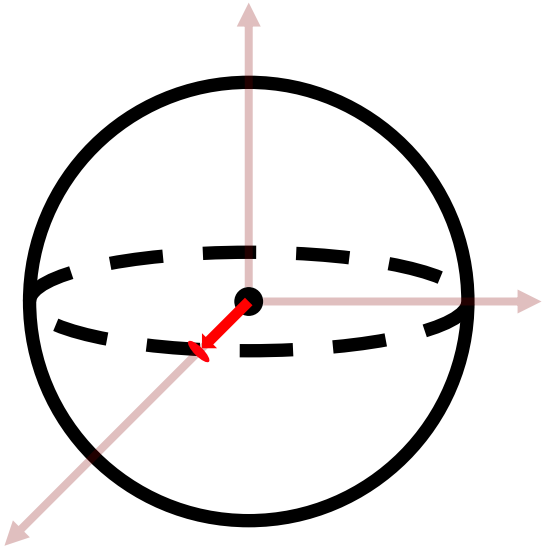


$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

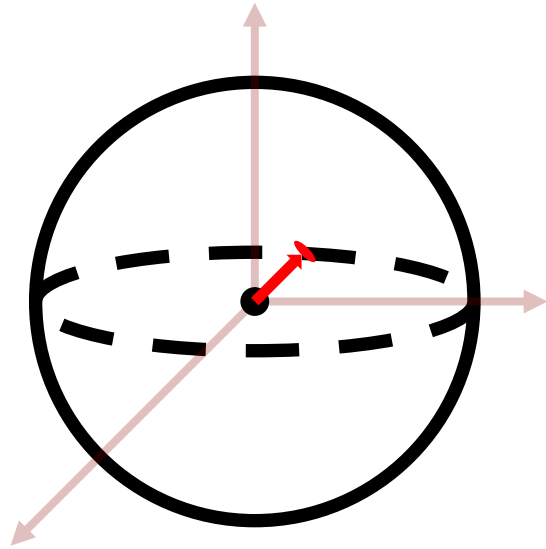
Are these the same?

$$|-\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

Common states in quantum computing



$$P(0) = \frac{1}{2} \qquad P(1) = \frac{1}{2}$$



$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Are these the same?

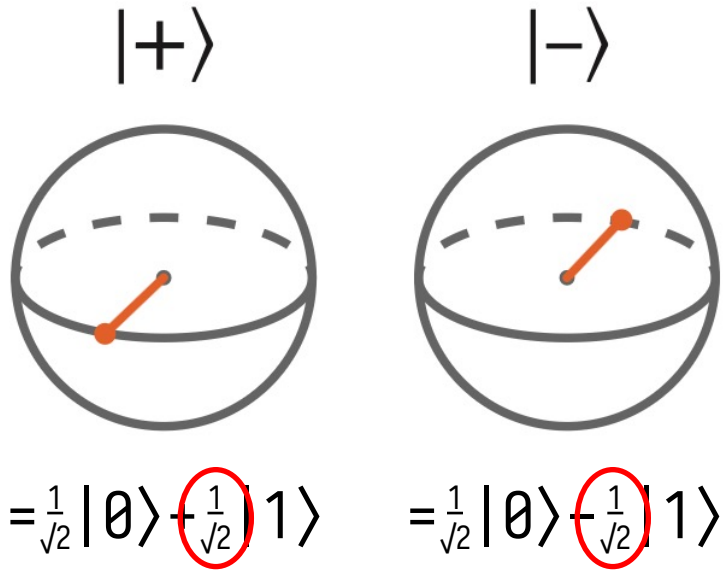
No

$$|-\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle$$

Amplitude and phase

A way to express quantum superposed state

(They are just how qubits are represented)



Amplitudes affect how they are measured

Phases do not affect how they are measured, but affect interference (they are *hidden!*)

$$\frac{1}{\sqrt{2}} \cdot 1 \qquad \frac{1}{\sqrt{2}} \cdot (-1)$$

Interference

The phenomenon that
qubits' phases and amplitudes
influence how qubits are measured.

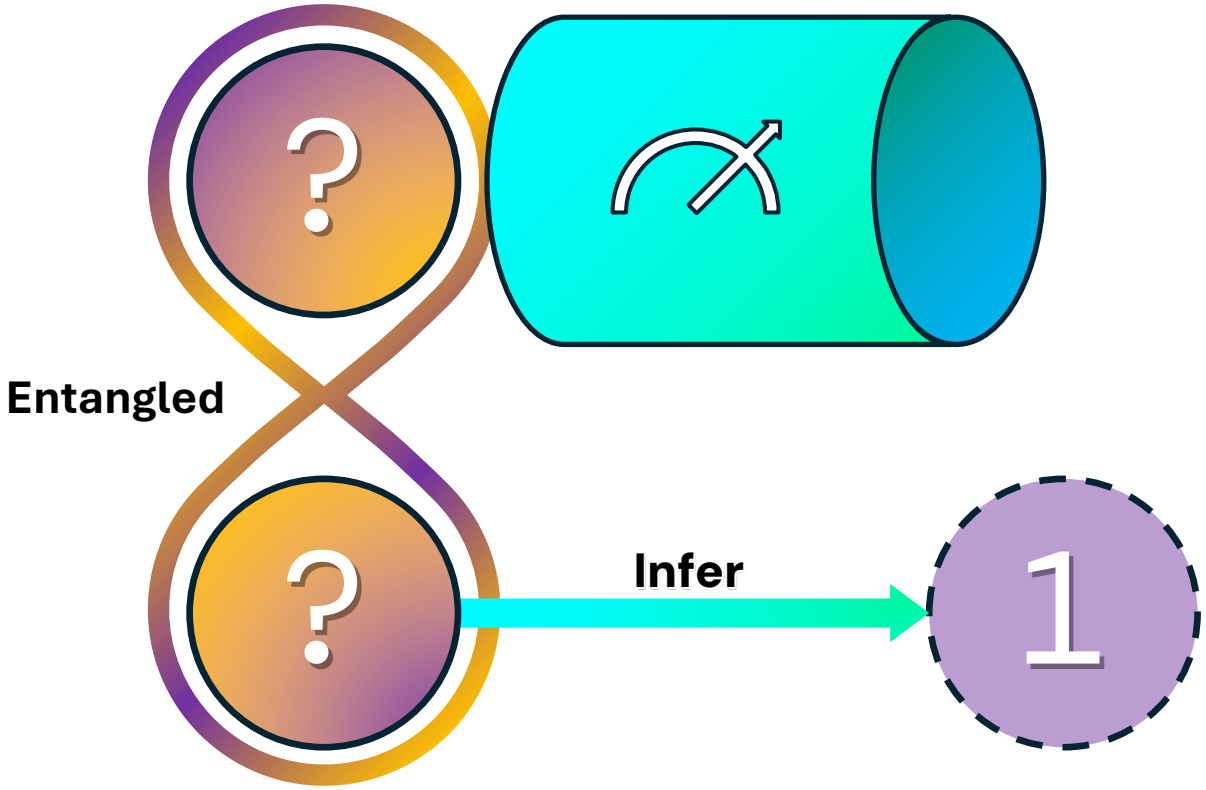
Interference is used to amplify or de-amplify
qubit states to obtain desired outcomes.

Visualizing how phases and amplitudes
propagate over a circuit is an interesting problem.

Entanglement

So far, we've only talked about single qubits.
But, it has limitations
to process more complex information.

When two qubits are entangled,
we can infer about one qubit's state
by looking at the other qubit's state.



Specific outcome may depend on
how two qubits are entangled.

Entanglement

If entangled to be in the same state,
what are the probabilities?

Equal superposition



$P(|00\rangle) = ?$

$P(|01\rangle) = ?$

$P(|10\rangle) = ?$

$P(|11\rangle) = ?$

Entanglement

If entangled to be in the same state,
what are the probabilities?

Equal superposition



$$P(|00\rangle) = 0.5$$

$$P(|01\rangle) = 0$$

$$P(|10\rangle) = 0$$

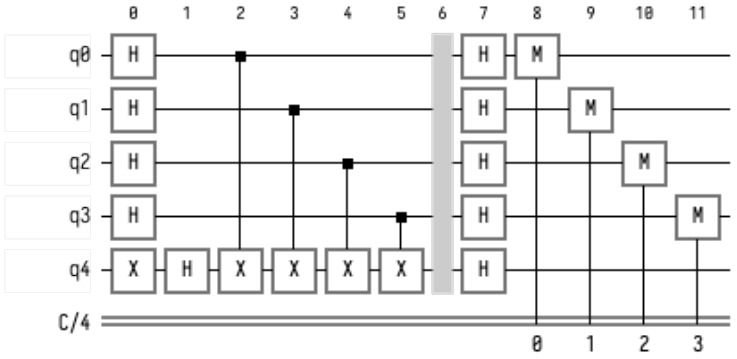
$$P(|11\rangle) = 0.5$$

Logical vs. physical (intuition)

What human thinks **What works on hardware**

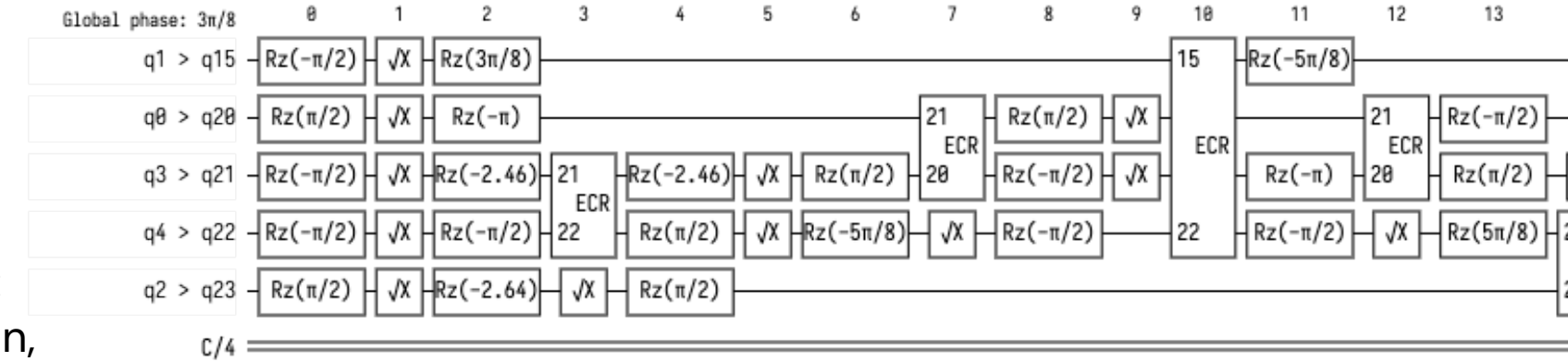
Logical and physical circuits

Logical circuits
Quantum programs
written by humans



Physical circuits
Quantum programs
compiled for physical machines

Compilation relies on many factors:
fidelity, qubit connectivity, defection,
resource sharing, etc.



Logical and physical qubits

Logical qubits
Qubits that we suppose as information units when we write programs.

Physical qubits
A single logical qubit is realized as a group of qubits in physical machines, involving some error-correcting qubits.

How to visualize this relationship is a complicated problem.
Why? There is a "time" dimension of how this works over a program.

Logical and physical gates

Logical gates
Human-written gates



Physical gates
Gates compiled for physical machines, often by decomposing logical gates into the several gate types that are supported by a given hardware machine.



No Cloning Theorem

A quantum state cannot be copied.

This property makes it difficult to save quantum states and reuse them.

Part 1

Introducing Quantum Computing

Topics covered

~~Quantum computers—qubit, gate, measure~~

~~Superposition, Bloch sphere, and entanglement~~

Shor's algorithm

Search algorithm (oracle-based)

Optimization problems

Quantum error correction (and mitigation)

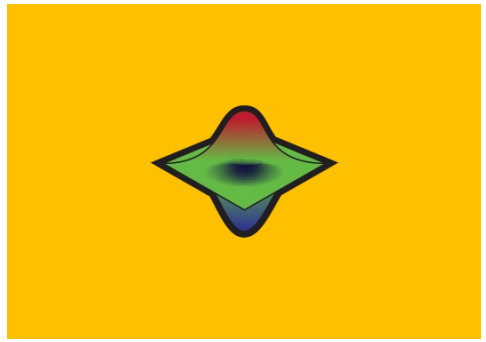
Key application areas

Security



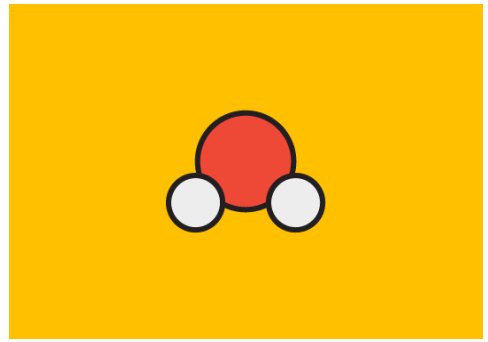
Providing a way to assess the strength of encryption methods

Physics



Simulating how quantum particles behave

Chemistry



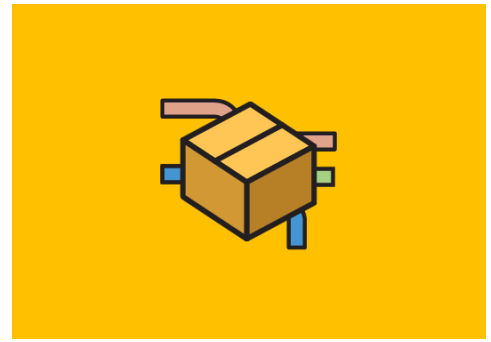
Estimating energy and wave functions of chemical molecules

Medicine



Discovering novel medicines

Logistics



Searching optimal schedules and routes for logistics

Furthermore…

- Telecommunication
- Quantum machine learning*
- Material science
- Stock portfolio optimization

Quantum cryptography

Shor's algorithm—for cryptography

Modern cryptography relies on large prime numbers
because it is highly costly to find prime factors (2048-bit = 2^{2048}).

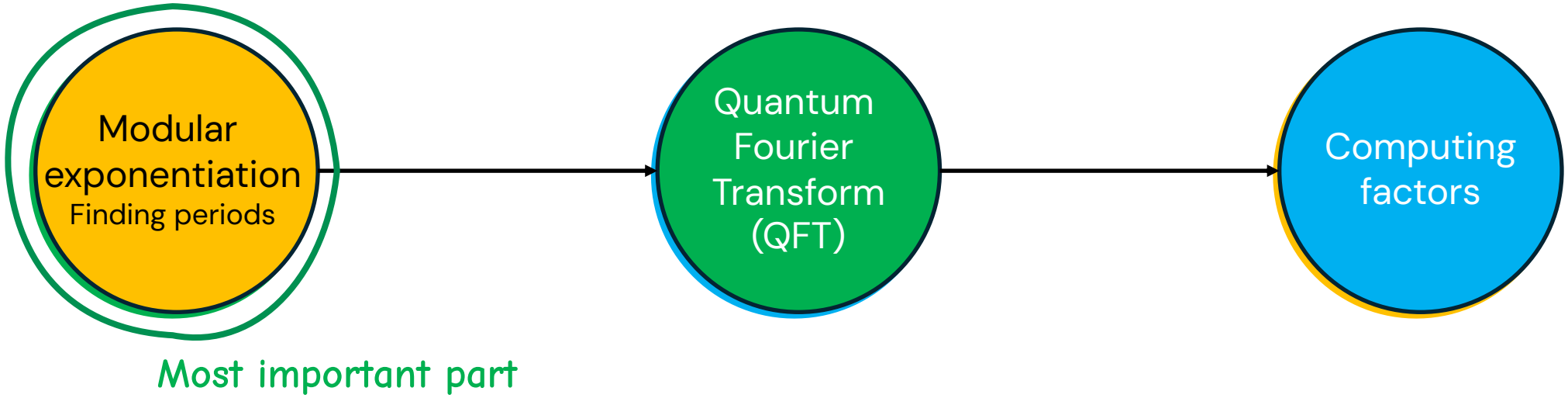
Shor's algorithm is for finding the prime factors of an integer.

RSA-2048 (modern standard)
300 trillion years with modern classical computers
➔ a few days with 26,000 physical qubits [Cain et al., 2026]

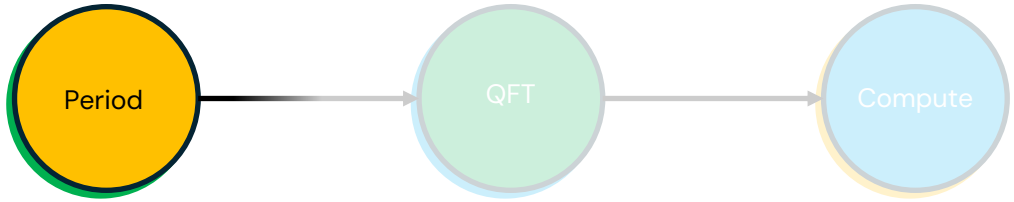
Shor's algorithm—for cryptography

In the essence, it is about guessing a number (as a factor) and verifying it.

Shor's algorithm consists of three parts:



Shor's algorithm—Part 1



Settings

a = the number we are guessing (given)

r = the period of modular exponentiation function (iterated)

N = the number to factor

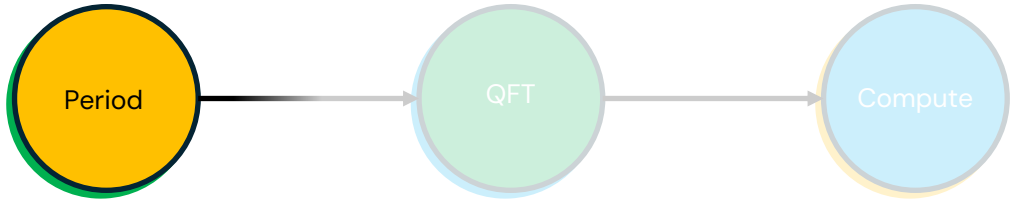
We want to find the smallest positive integer **r** such that

$$a^r \equiv 1 \pmod{N}$$

Why?: hints for factorization

(**a** to the power of **r**) and 1 have the same remainder when divided by **N**

Shor's algorithm—Part 1



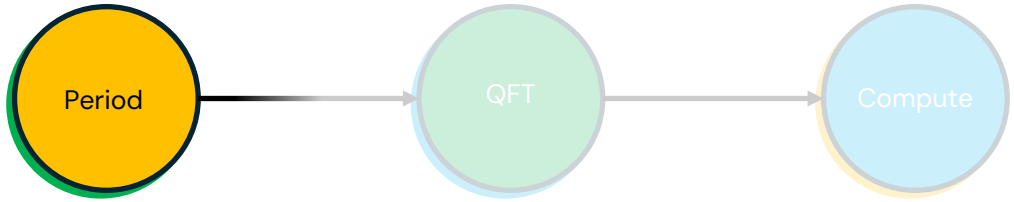
$$a^r \equiv 1 \pmod{N}$$

Because a^r and **1** have the same remainder when divided by **N**

$(a^r - 1)$ can be divided by **N** and another integer.

$$N \times k = (a^r - 1)$$

Shor's algorithm—Part 1



$a^r \equiv 1 \pmod{N}$

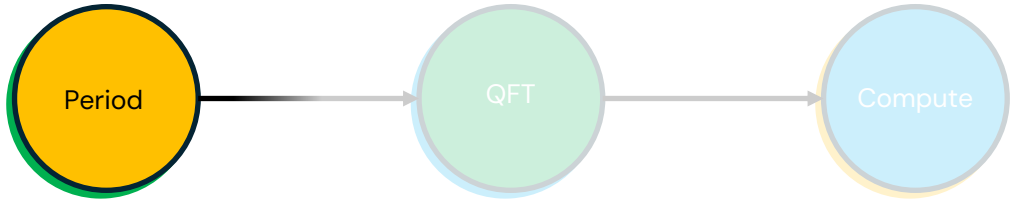
$$N \times k = (a^r - 1) = (a^{r/2} - 1)(a^{r/2} + 1)$$

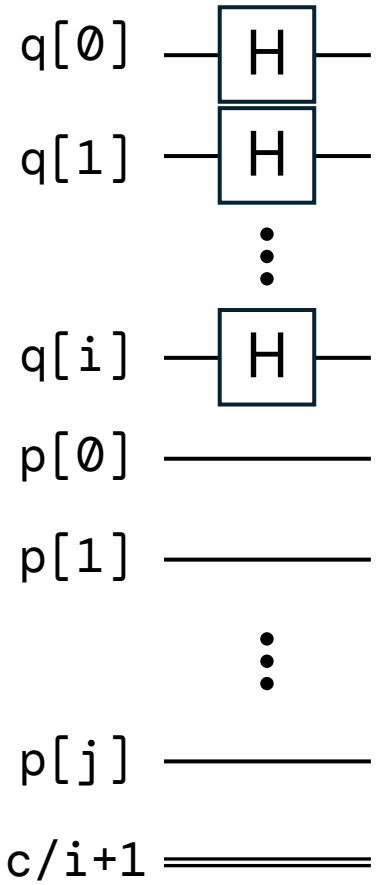
These numbers may not be factors of N,
but likely to be co-factors.

In other words,
we can recursively repeat this process.

Because we want integers,
we want r to grow exponentially by 2.

Shor's algorithm—Part 1

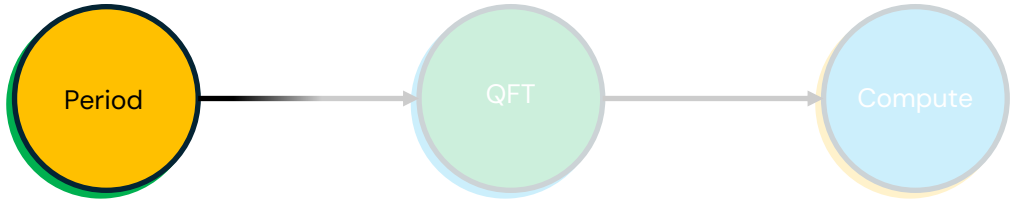




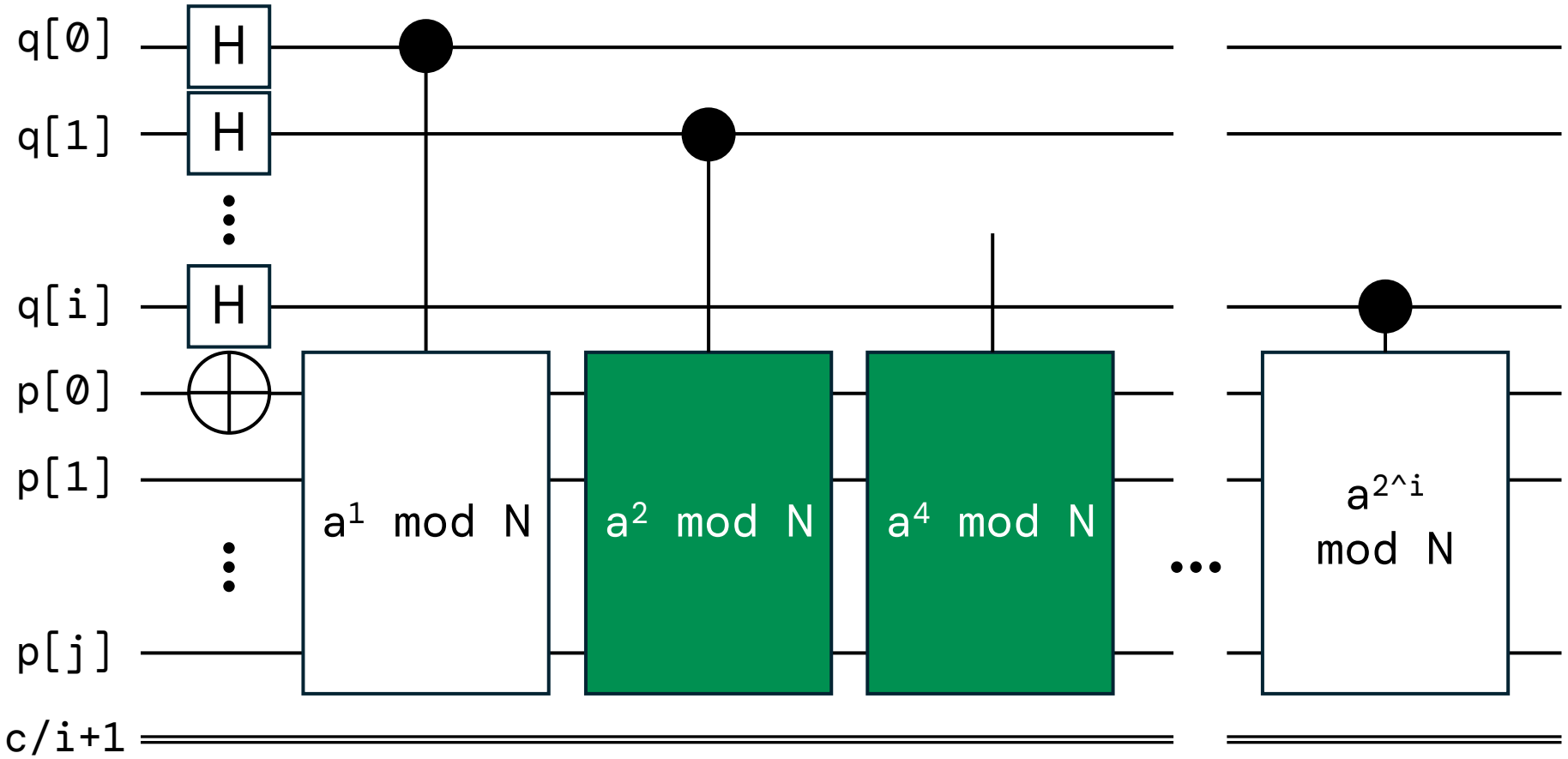
(Simply put)

If q[0...i] represents |0...0⟩=|X⟩,
(e.g., |0110⟩=|6⟩)

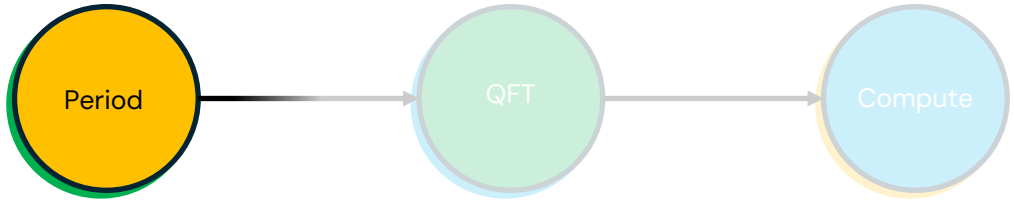
Shor's algorithm—Part 1

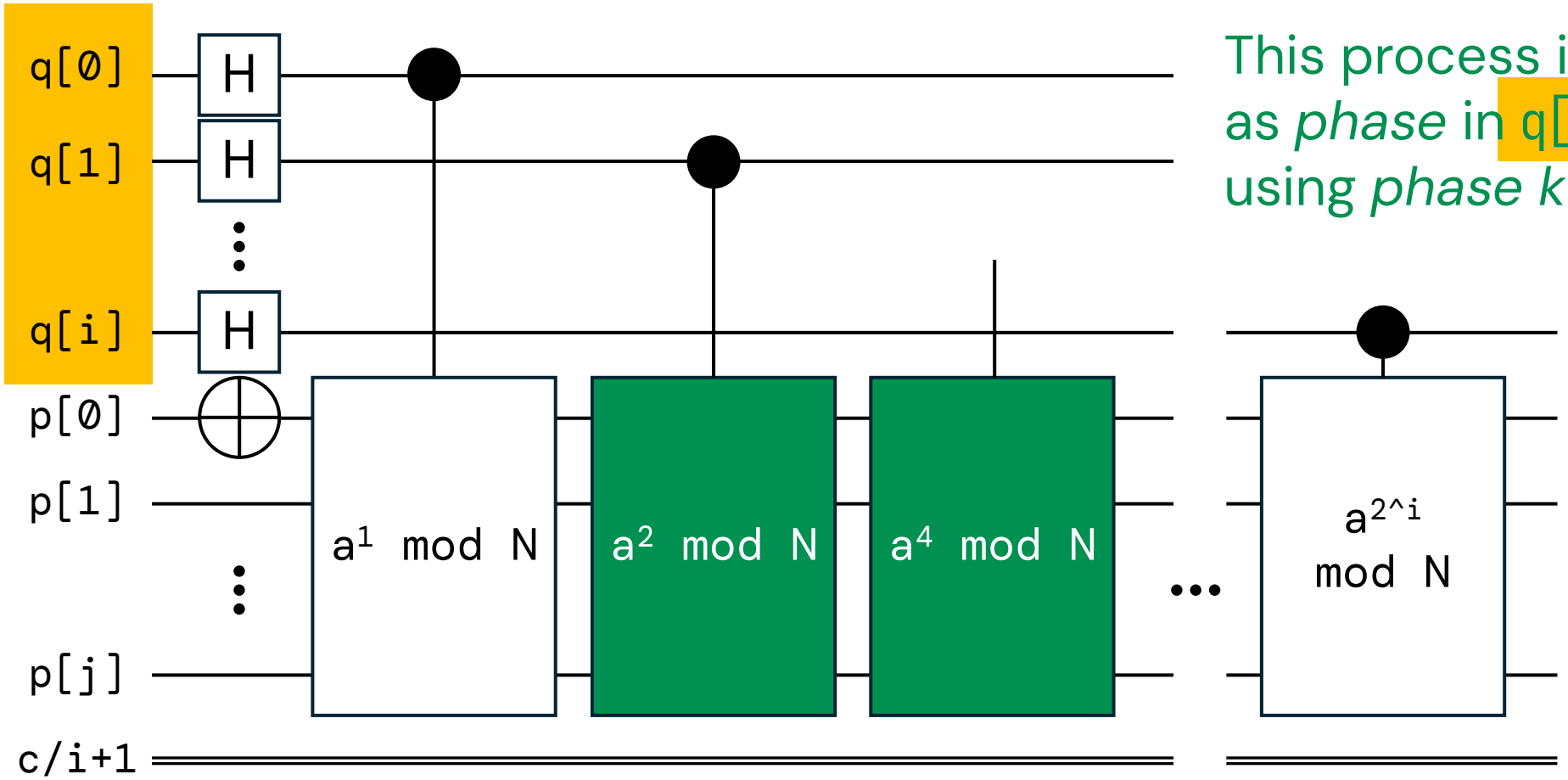


Then, runs $\mathbf{a^{2+4=6}}$ mod **N**.



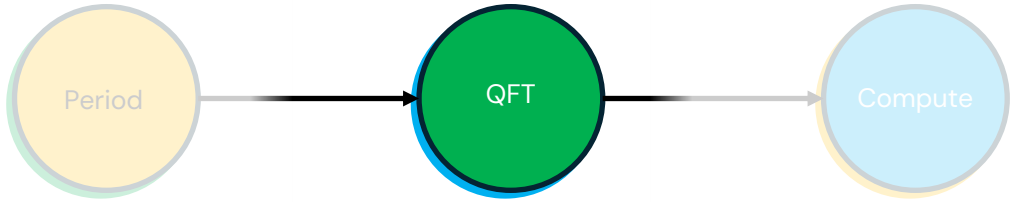
Shor's algorithm—Part 1

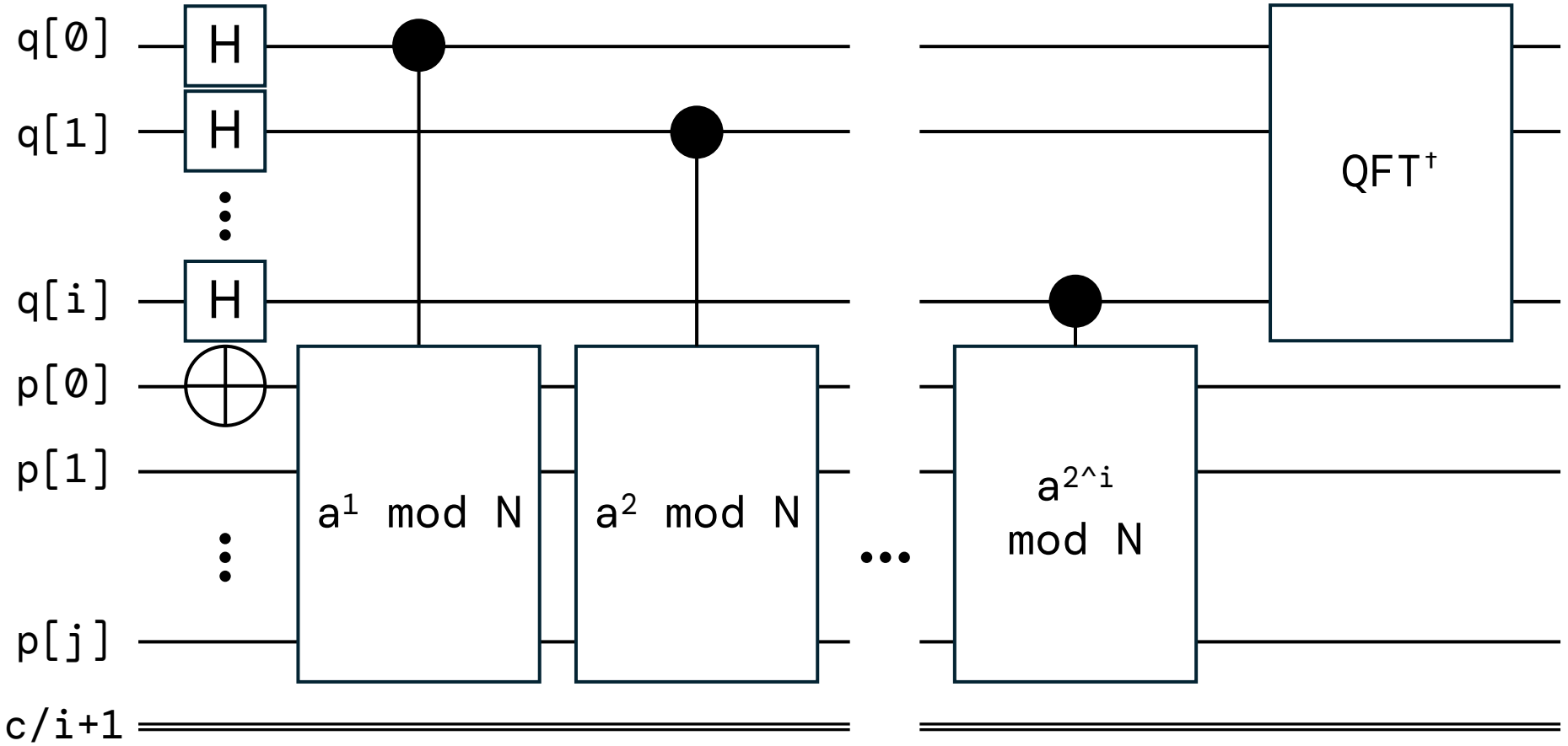




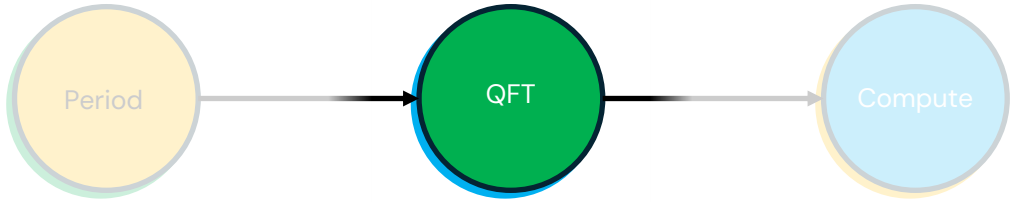
This process is recorded as *phase* in `q[0...i]` using *phase kickback*.

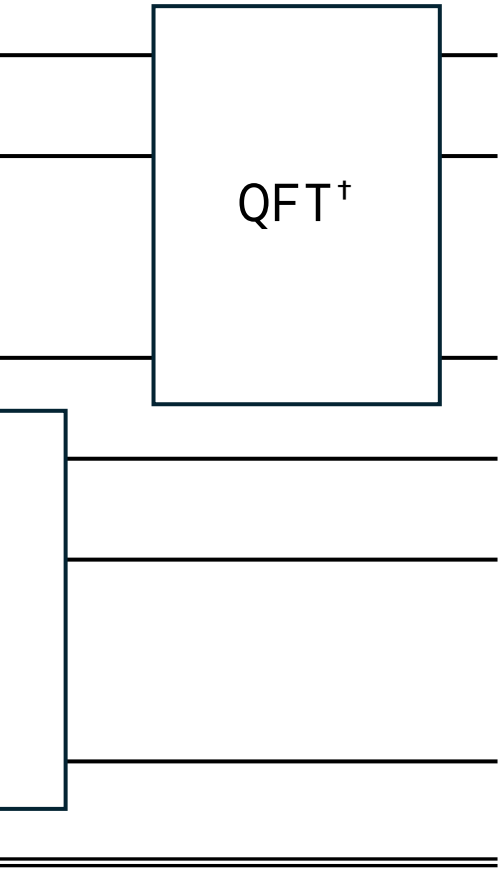
Shor's algorithm—Part 2





Shor's algorithm—Part 2

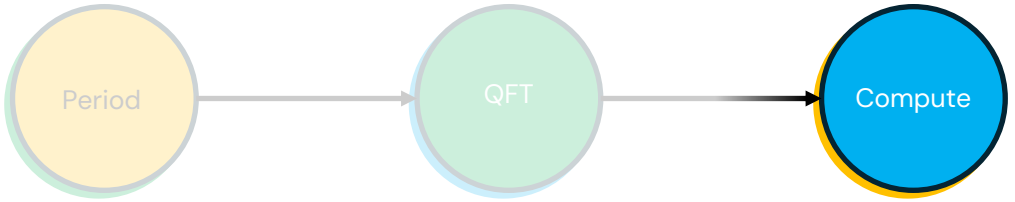


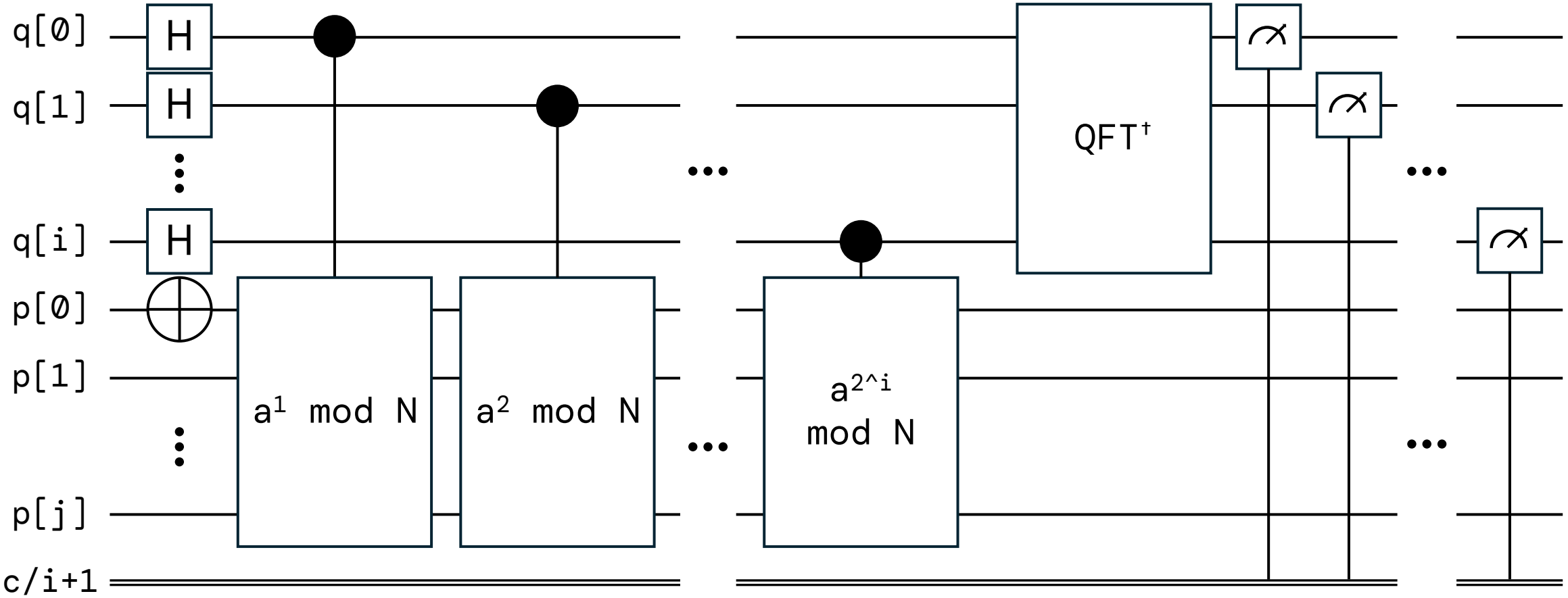


The quantum analogue of the discrete Fourier transform.

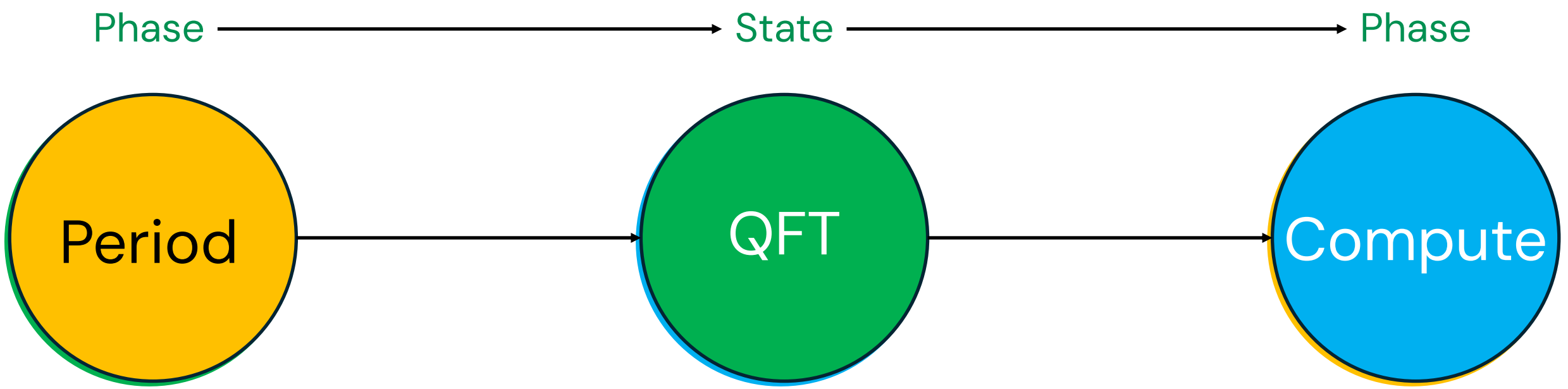
= revealing the hidden "phases" as states

Shor's algorithm—Part 3





Shor's algorithm—Part 3



Shor's algorithm—Example

a = 2, the number we are guessing (given)

r = the period of modular exponentiation function (iterated)

N = 15, the number to factor

Estimated phases

	Register Output	Phase		Phase Fraction	Guess for r
0	00000000(bin) = 0(dec)	0/256 = 0.00	0	0.00	0/1	1
1	01000000(bin) = 64(dec)	64/256 = 0.25	1	0.25	1/4	4
2	10000000(bin) = 128(dec)	128/256 = 0.50	2	0.50	1/2	2
3	11000000(bin) = 192(dec)	192/256 = 0.75	3	0.75	3/4	4

Source: Qiskit

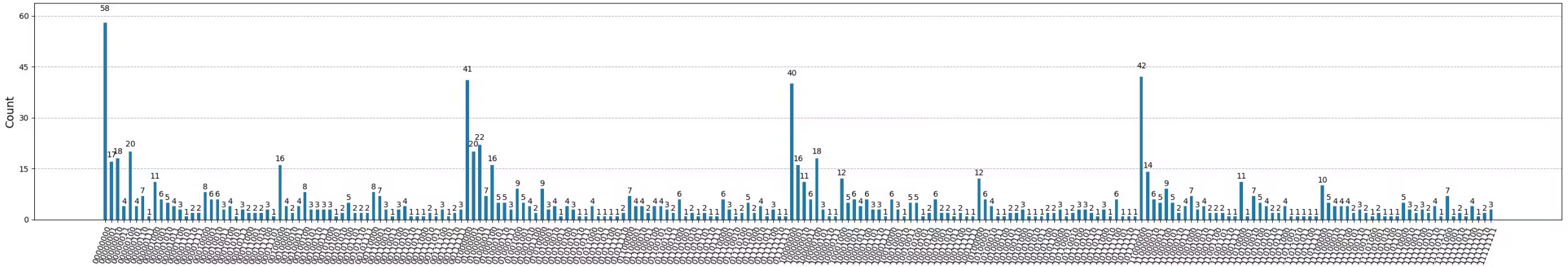
Strongest guess for **r = 4**

$a^{r/2} - 1 = 4 - 1 = 3$

$a^{r/2} + 1 = 4 + 1 = 5$ 15 = **3 • 5**

Shor's algorithm & visualization

How can we help linking this periodic information
more intuitively to the factors that we are looking for?



Source: Qiskit

This isn't only about Shor's algorithm.
This is relevant to many of quantum algorithms.

Shor's algorithm—Cautions

But, there are some constraints currently.

1. We don't know a generalized method to encode modular exponentiation.

2. To effectively factorize application-scale large integers,
   we need a few thousands of qubits (current commercial availability: ~150).

Quantum roadmap

Source: IBM

We are going to see QPUs with 2K+ qubits in 2033+

Part 1

Introducing Quantum Computing

Topics covered

~~Quantum computers—qubit, gate, measure~~

~~Superposition, Bloch sphere, and entanglement~~

~~Shor's algorithm~~

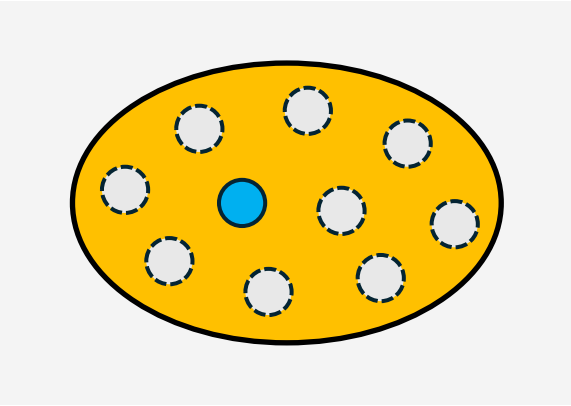
Search algorithm (oracle-based)

Optimization problems

Quantum error correction (and mitigation)

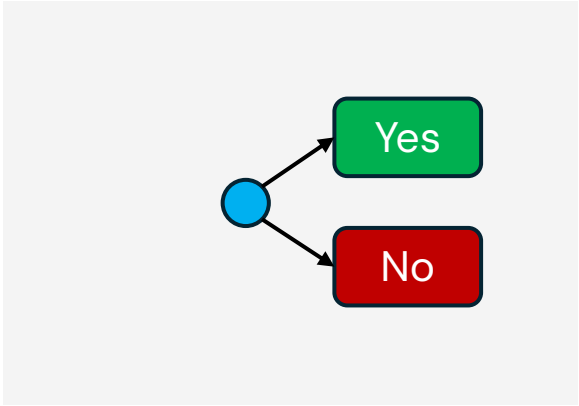
Theoretic classification of problems

Search



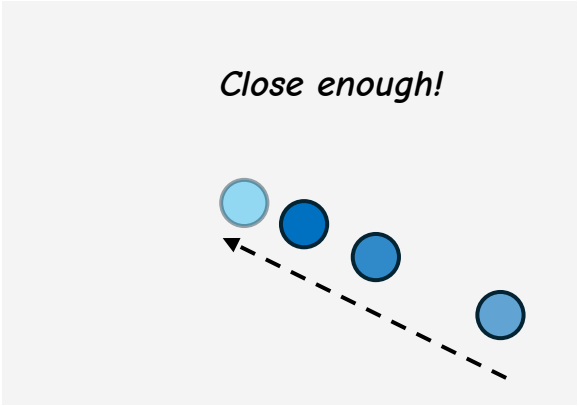
Find an answer

Decision



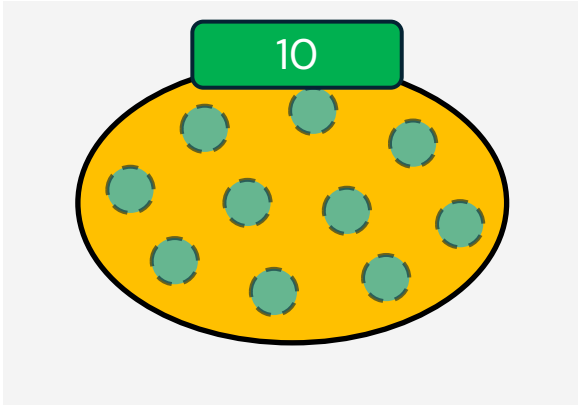
Determine an answer

Optimization



Approximate an answer

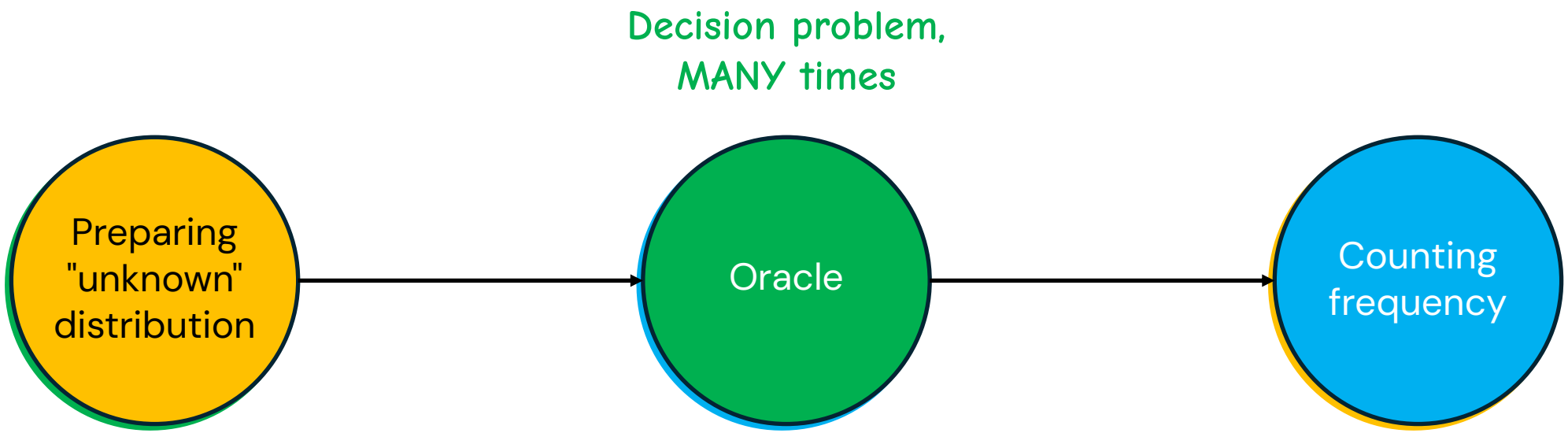
Counting



Count # of answers

Quantum search problems builds on top of decision problems.

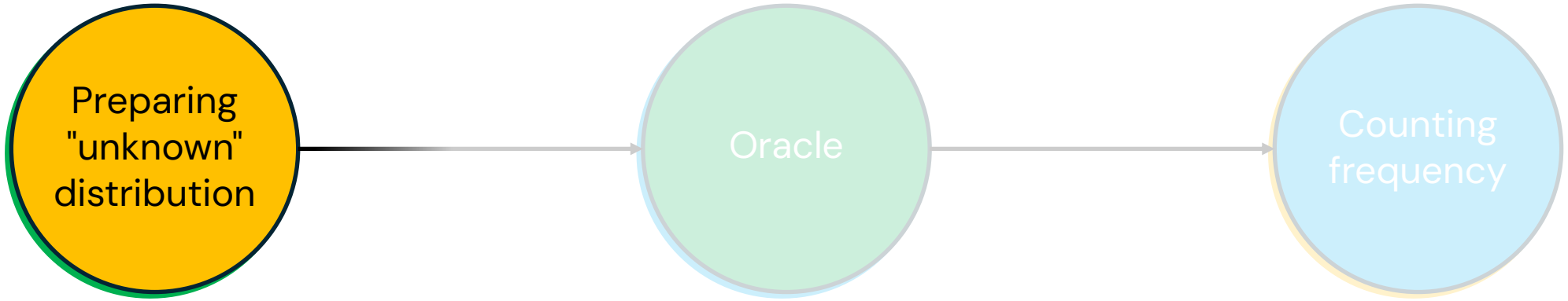
Grover's algorithm in a nutshell



Grover's search can solve problems
that we can constitutes the correctness of answers
but it is intractable to search correct candidate answers
(e.g., satisfiability, hashed key recovery).

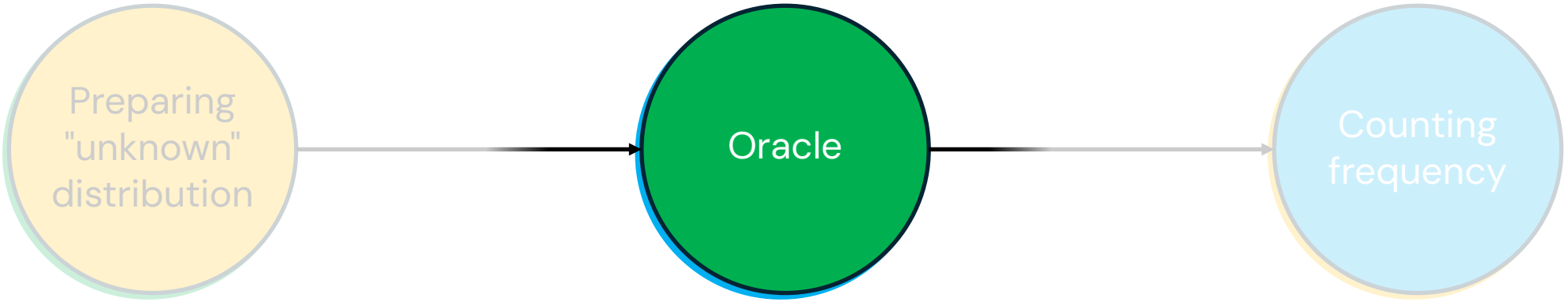
Superposition as randomization

By uniformly superposing qubits (=a bunch of Hadamard gates)
we can prepare an "unknown" distribution (nothing is assumed here yet).



Oracle: amplify toward correct answer

Depending on different problems,
there are specific ways to encode oracles.

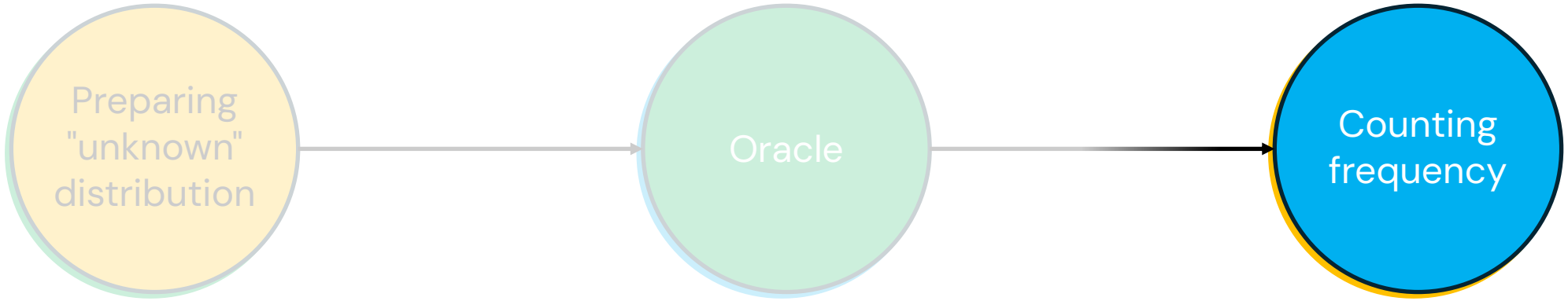


Recover the amplitude by measuring

High-frequency candidates are likely answers.

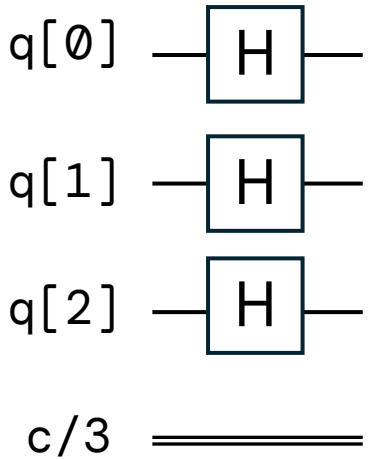
Needs to run $\frac{\pi}{4}\sqrt{N}$ times

$N = 2^{n(qubits)}$ (the number of items being searched)

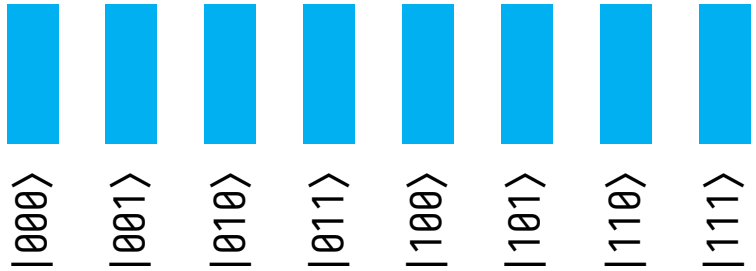


Example

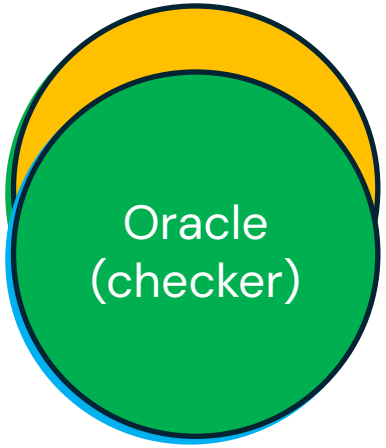
Preparing "unknown" distribution

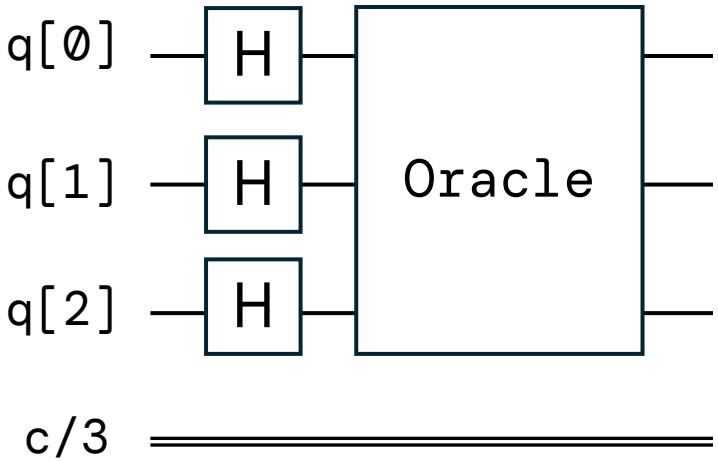


Uniform distribution



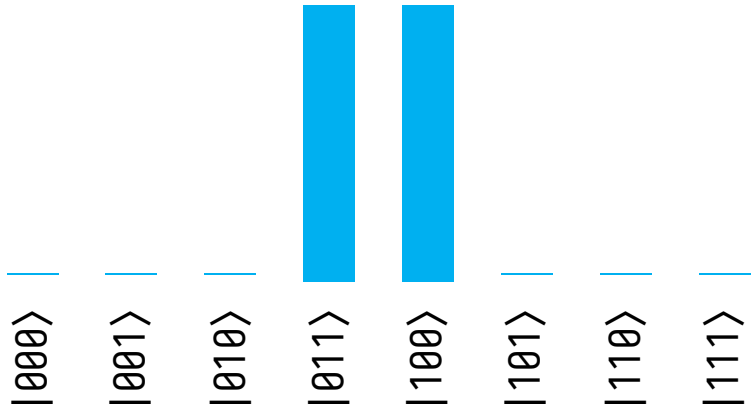
Example





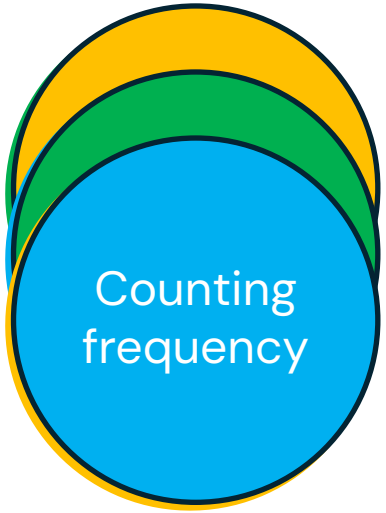
Oracles can be defined as
- a hash function
- a logic operator
- …

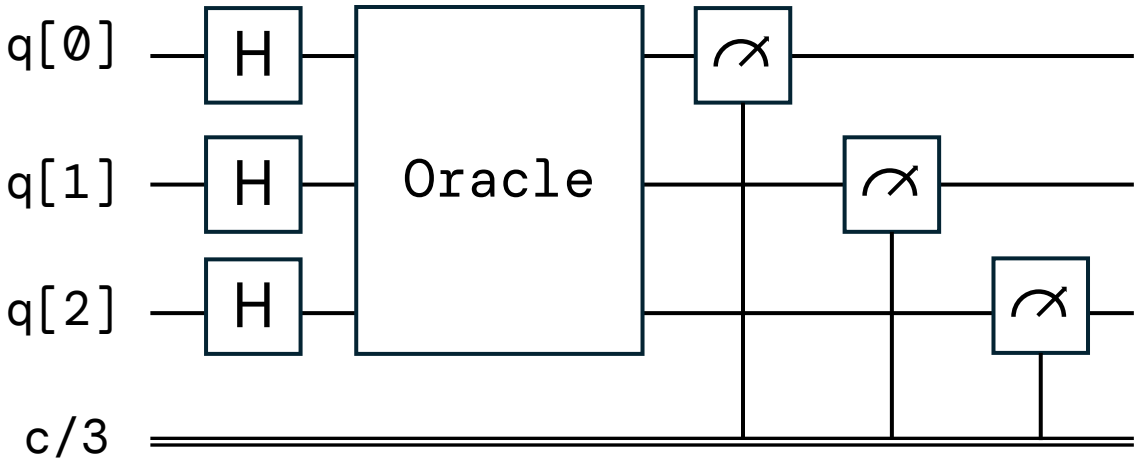
Theoretical frequency



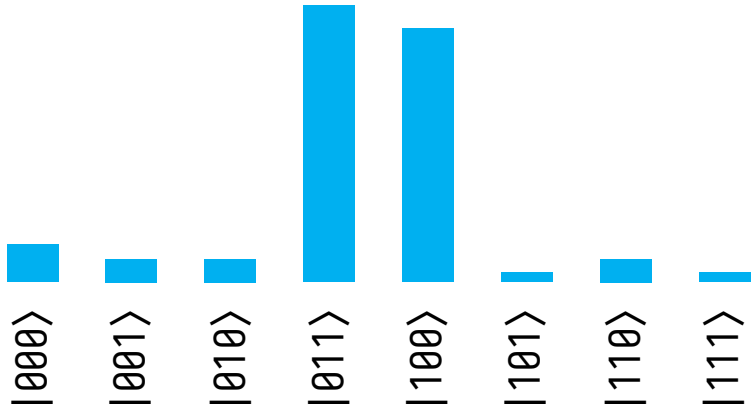
But, we cannot calculate them in advance in useful scenarios because it will be intractable.

Example





Actual measurement outcome



Grover's algorithm—Notes

We need more work on useful oracle methods for application-scale problems.

Currently, many tutorial examples include very simple cases (e.g., string match).

To do problems like unstructured database search,
oracles must be able to associate each candidate value
with the corresponding index (represented as qubits).
This needs quantum RAMs (QRAMs).

Part 1

Introducing Quantum Computing

Topics covered

~~Quantum computers—qubit, gate, measure~~

~~Superposition, Bloch sphere, and entanglement~~

~~Shor's algorithm~~

~~Search algorithm (oracle-based)~~

Optimization problems

Quantum error correction (and mitigation)

Quantum Approximate Optimization Algorithm (QAOA)

A representative quantum optimization algorithm

It finds a certain string that maximizes (or minimizes) the cost.

String = a network, a Boolean function, …
Cost = utility, benefits, difficulty, …

Quantum optimization is about finding the ground state energy

Uhm... what? 😒

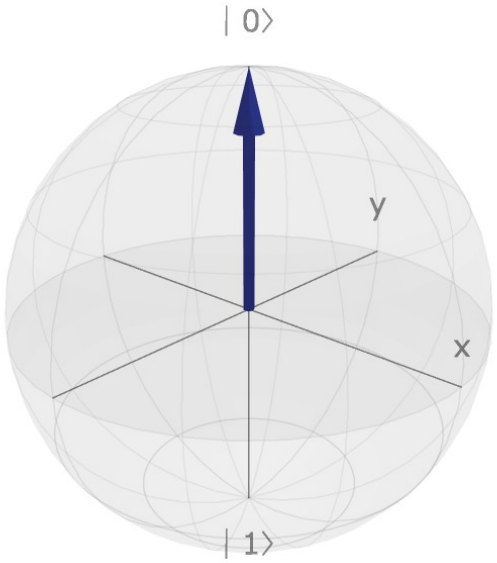
The unique energy level(s)
of a physical system
at the absolute zero temperature (0°K)

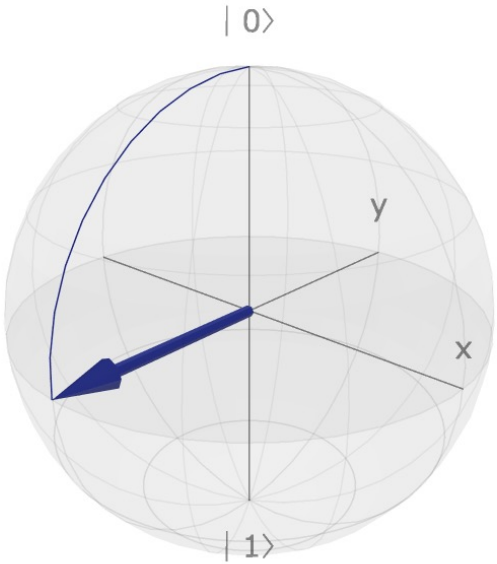
Hamiltonian
a matrix that expresses
the total energy of a physical system

Quantum optimization is about finding the ground state energy

Background knowledge 1
Some gates have parameters.

RX(π/2)
Rotate a qubit
by $\pi/2$ (90 degrees)
by the X axis.





Quantum optimization is about finding the ground state energy

Background knowledge 2 –Quantum Adiabatic Process

When a quantum system evolves slowly enough,

the system will adapt

to the environment's ground state energy

In our case (for quantum computing)…

If there are sufficient gaps between operations

the states of qubits will approximate

to those of the solutions

Quantum optimization is about finding the ground state energy

Background knowledge 2
–Quantum Adiabatic Process

When a quantum system evolves slowly enough,

the system will adapt

to the environment's ground state energy

In our case
(for quantum computing)…

If there are sufficient gaps between operations

the states of qubits

will approximate

to those of the solutions

Represented as gate parameters

Quantum optimization is about finding the ground state energy

Background knowledge 2 –Quantum Adiabatic Process

When a quantum system evolves slowly enough,

the system will adapt

to the environment's ground state energy

In our case (for quantum computing)…

If there are sufficient gaps between operations

the states of qubits will approximate

to those of the solutions

Our QPUs are cooled at 0°K.

An analogy to ground state energy

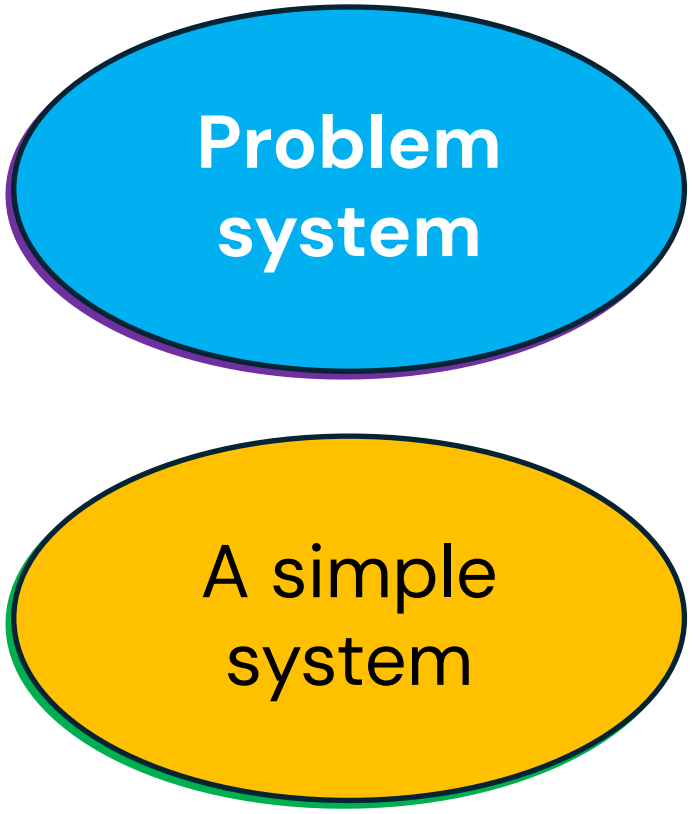
Problem = Physical system
Solution = Ground state

We express our problem as a physical system
in a way that its ground state
is the desired answer of the problem.

Questions so far?

How it works

1. Two systems (or Hamiltonians)



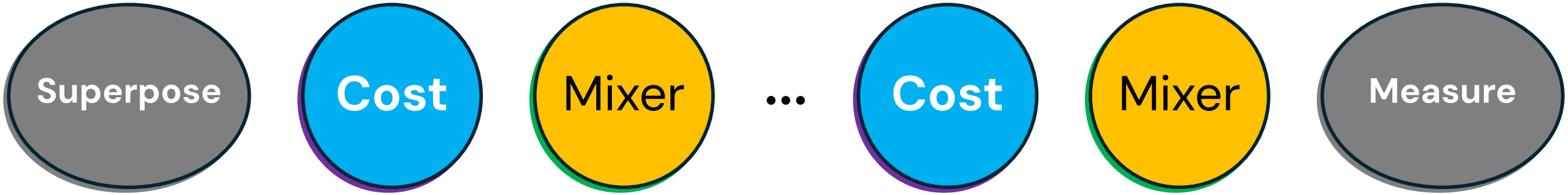
Its ground state = the solution (cost operator)

Without this,
the problem system fails to change due to some quantum mechanical properties (mixer operator)

https://quantumcomputing.stackexchange.com/questions/17555/whats-the-role-of-mixer-in-qaoa

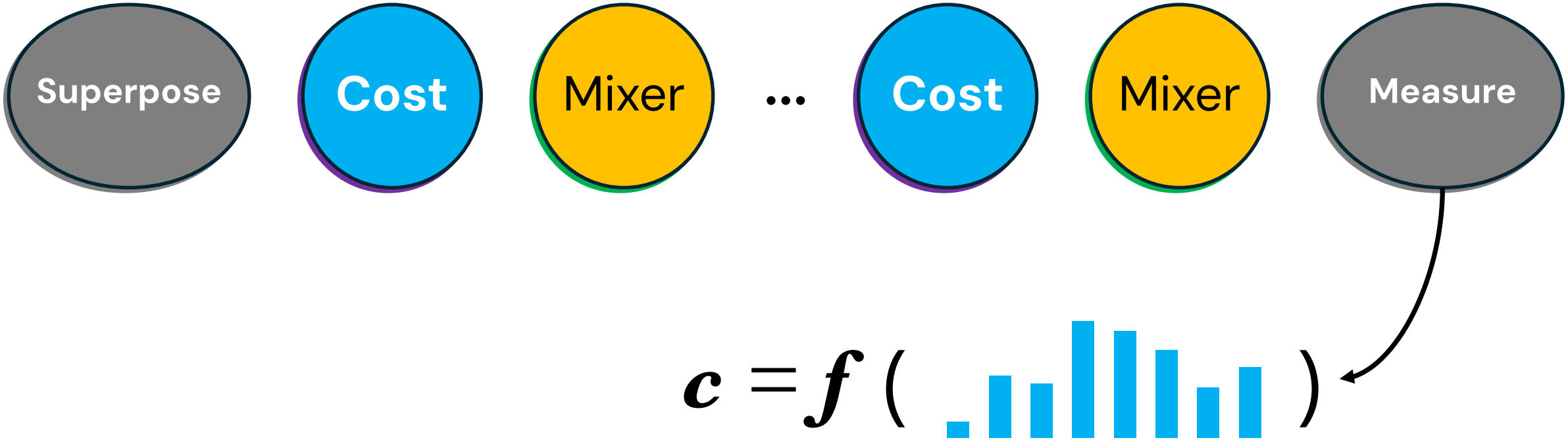
How it works

2. Iterate Cost and Mixer operators



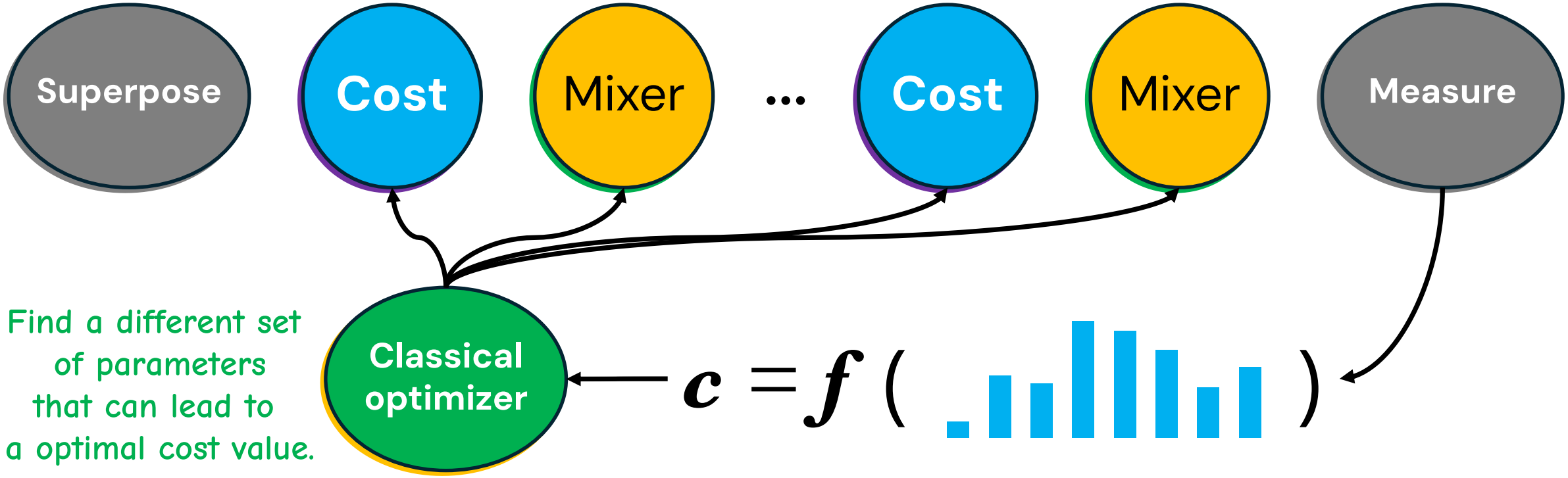
How it works

3. See how the desired "cost" is updated using the measured outcome



How it works

4. Using some classical optimizer (e.g., linear approximation; COBYLA), update the parameters



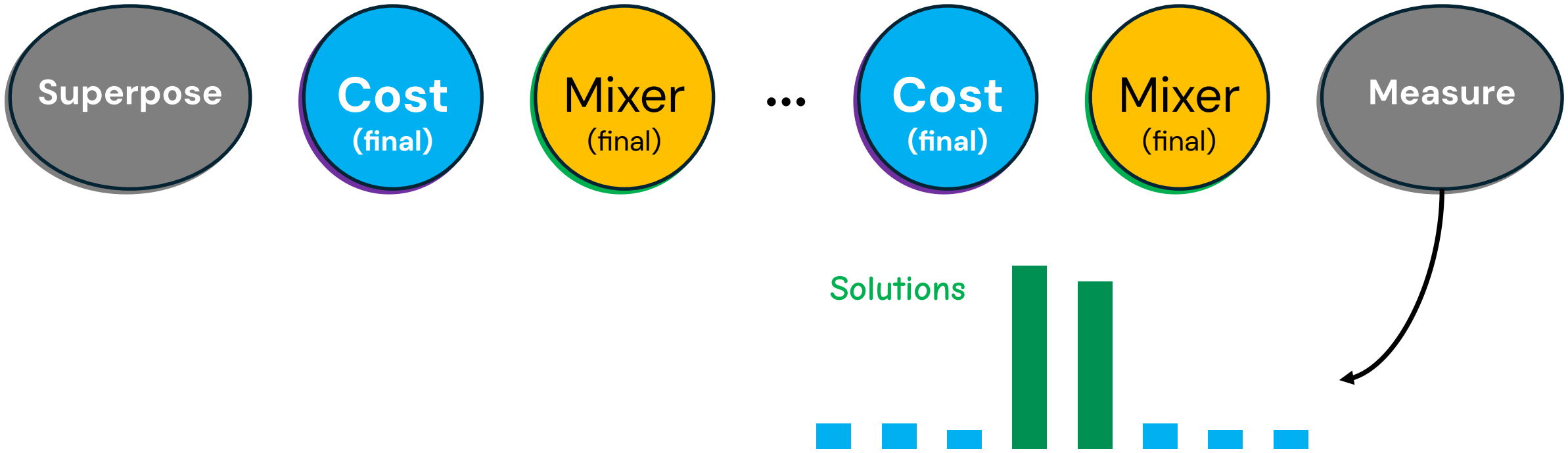
How it works

5. Repeat the process and
   get the final parameters as the cost converges

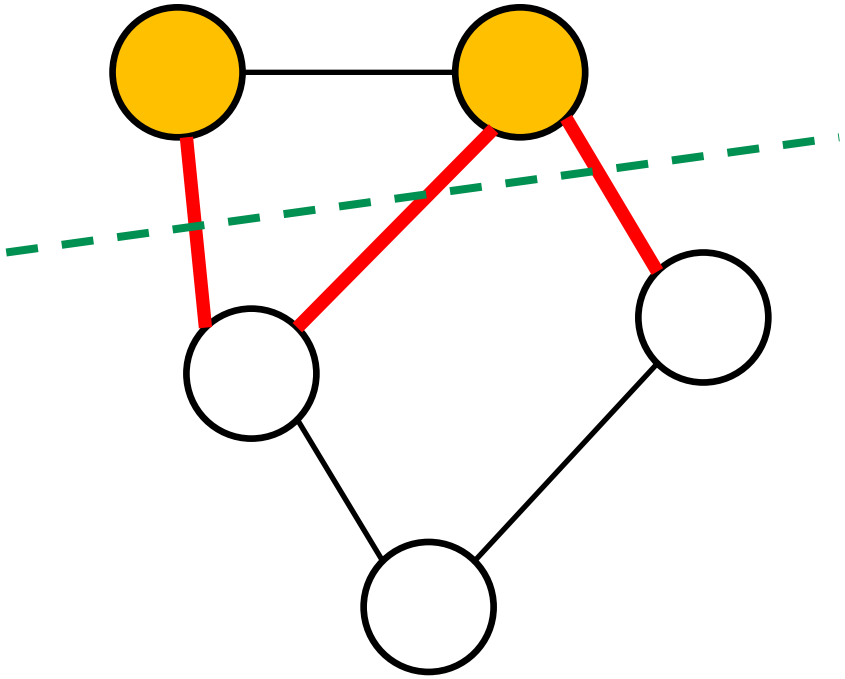
See handout

How it works

6. Run the circuit with the final parameters
   and then analyze the measurement outcome to find the solution



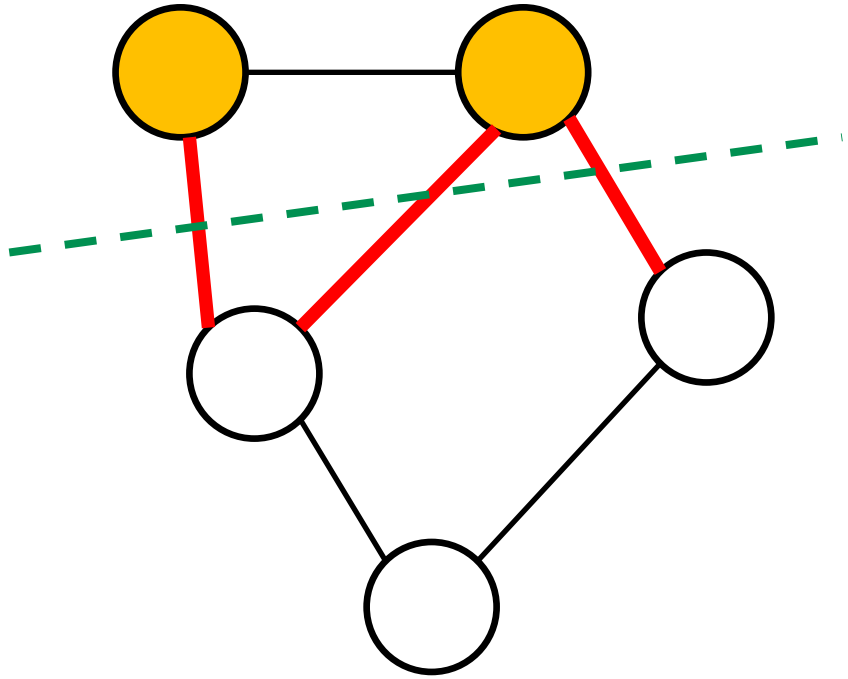
Classroom example: MAXCUT



Finding a bi-partition of nodes that maximizes the number of edges crossing between the two groups (="cuts")

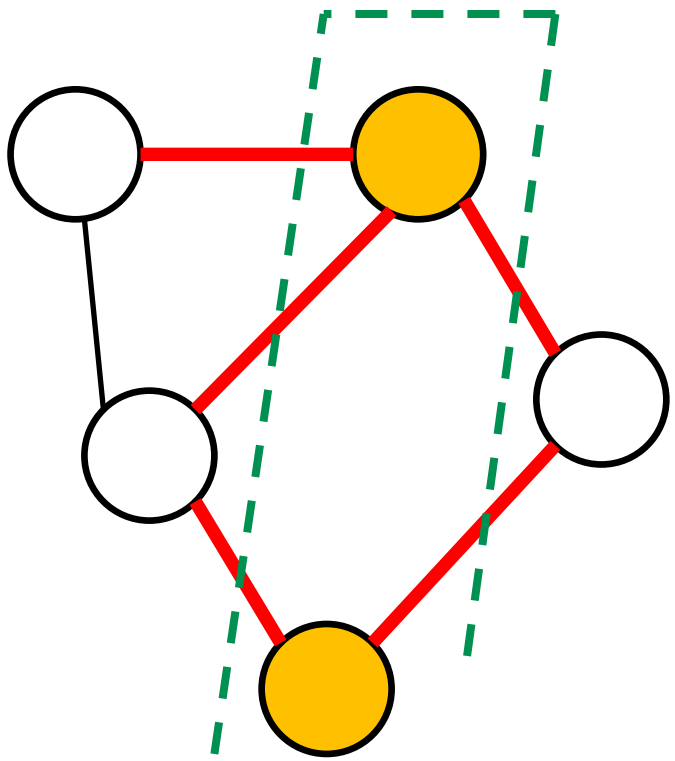
There is no known polynomial time solution for this problem (NP-hard).

Classroom example: MAXCUT



CUTs = 3

Classroom example: MAXCUT

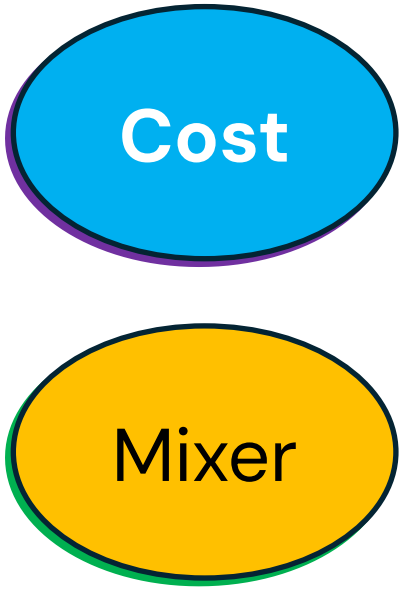


CUTs = 5

(Solution)

Classroom example: MAXCUT

Finding a bi-partition of nodes that maximizes the number of edges crossing between the two groups (="cuts")



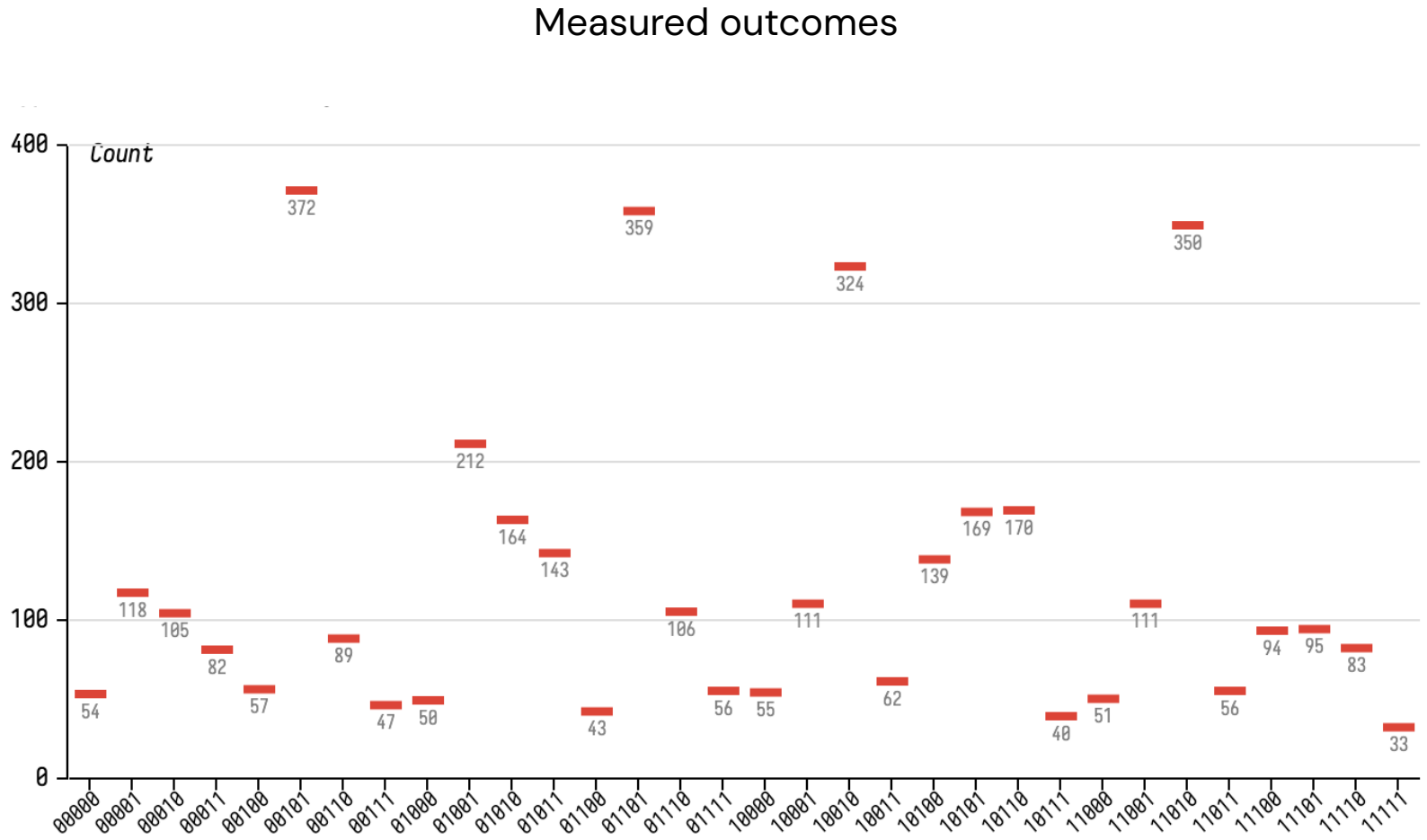
Edges in a graph

A bunch of X gates

cost *function*

The number of "cuts"

Visualizing the outcomes

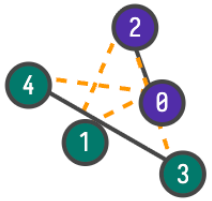


Visualizing the outcomes

Problem-specific visualizations

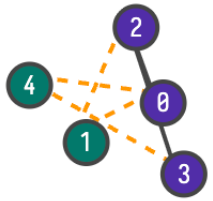
Likely cut #1

- Bitstring: 10100
- Count: 372
- Cost: -4
- # Cuts: 4
- Cuts: (1, 3, 4) / (0, 2)



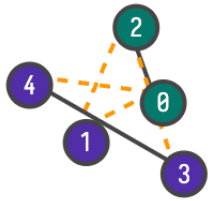
Likely cut #2

- Bitstring: 10110
- Count: 359
- Cost: -4
- # Cuts: 4
- Cuts: (1, 4) / (0, 2, 3)



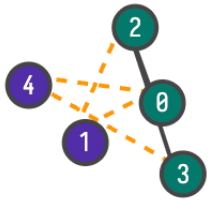
Likely cut #3

- Bitstring: 01011
- Count: 350
- Cost: -4
- # Cuts: 4
- Cuts: (0, 2) / (1, 3, 4)



Likely cut #4

- Bitstring: 01001
- Count: 324
- Cost: -4
- # Cuts: 4
- Cuts: (0, 2, 3) / (1, 4)



More applications of QAOA

Logistics

Telecommunication

Stock portfolio

Variational Quantum Eigensolver (VQE)

We are still interested in ground state energy.
Often used for finding
the actual ground state energy of a molecule.

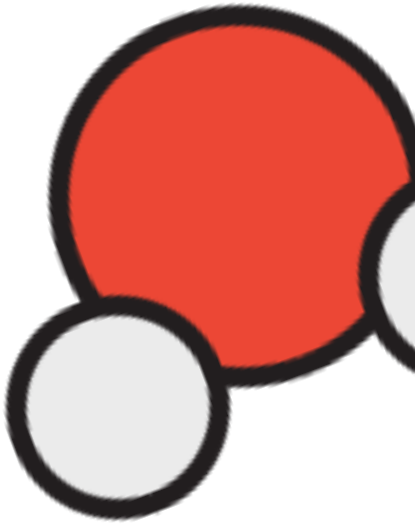
We keep using adiabatic procedures.



Variational Quantum Eigensolver (VQE)

At the end of the day
we want the optimized "parameters" out of this process
because those parameters represent the molecule's wavefunction
that exhibit its ground state energy.

Why does this matter?
It tells us about how molecules behave
(when they react with others; when they stay stable, etc.),
which is super important in chemistry.



Part 1

Introducing Quantum Computing

Topics covered

~~Quantum computers—qubit, gate, measure~~

~~Superposition, Bloch sphere, and entanglement~~

~~Shor's algorithm~~

~~Search algorithm (oracle-based)~~

~~Optimization problems~~

Quantum error correction (and mitigation)

Noisy intermediate-scale quantum computing (NISQ)

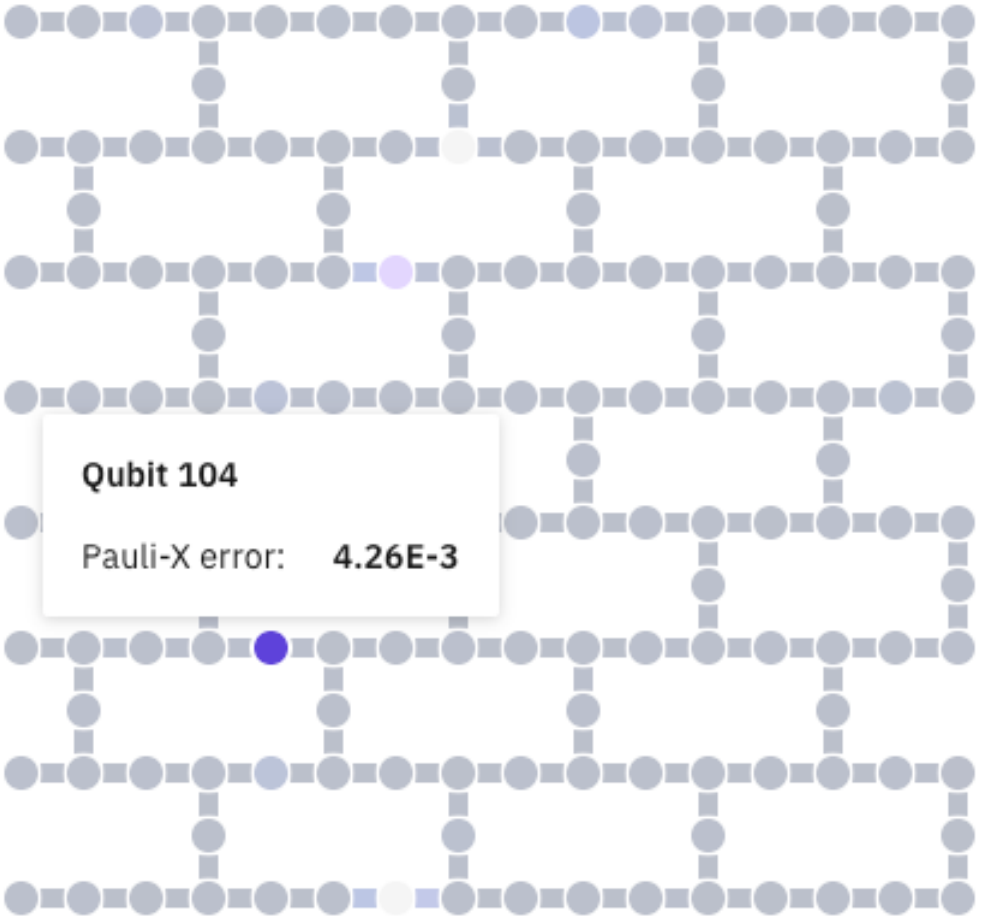
QPUs with not too many qubits (~1,000)
Not yet fault-tolerant

Currently, widely available commercial quantum computers have about 25-150 qubits.

They exhibit non-negligible amount of errors.

IBM Heron3 machine

Collected Aug 14, 00:00 am PT



IBM-Boston machine

(not in Boston, it's just how IBM names machines.)

156 qubits

Calibrated (estimated)
X gate error rate: 0.00426
4.3 errors every 1,000 operations.

5-node MaxCut QAOA has
a few thousand physical operations.

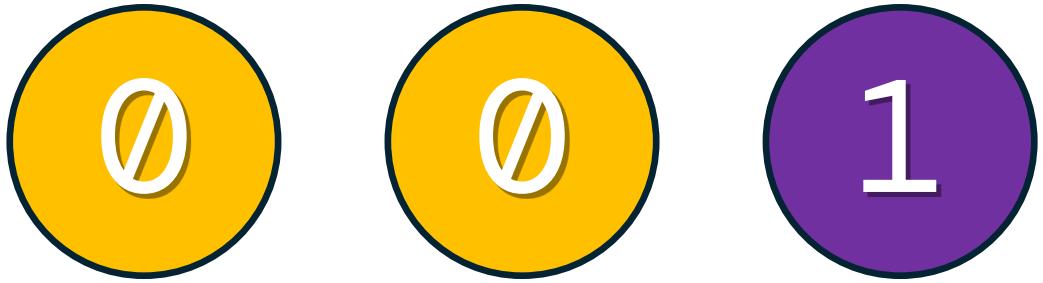
We need to correct errors, but it's not new in quantum

Classical bits also have errors, just a lot fewer times.
Less than 1 out of trillion times.

But classical computers still fix those errors just in case.

Error correction = Majority vote (Repetition code)

Run the same operation N times (typically 3 is enough).
Then, the majority outcome is the correct value.



These are bit-flip errors.

Quantum bit flip error correction

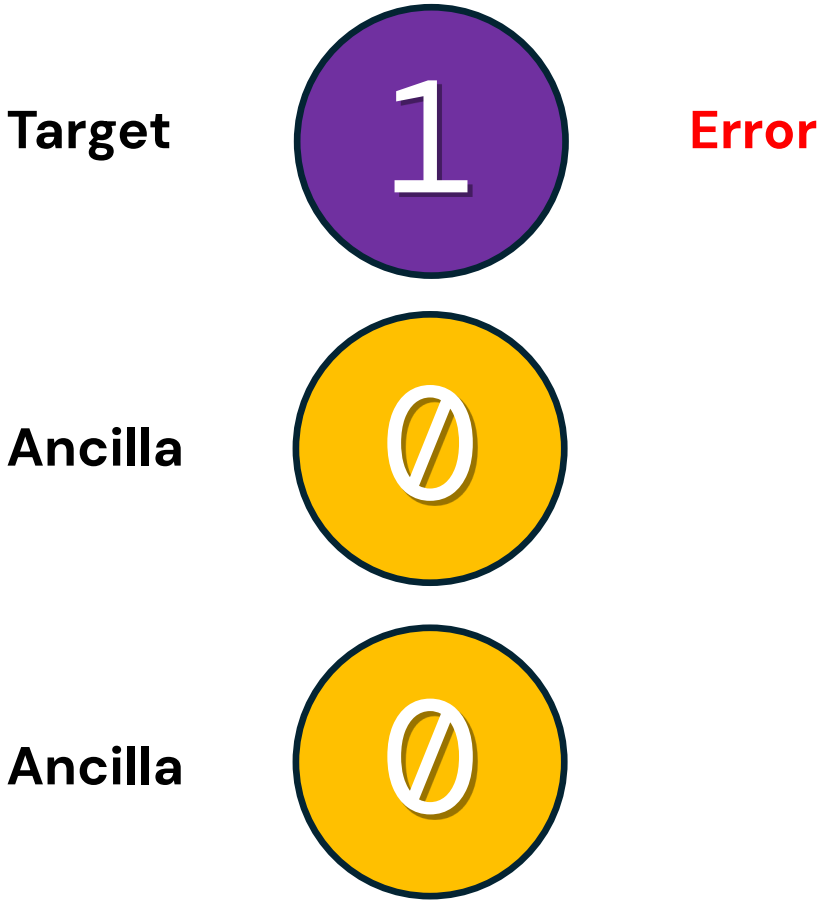
Let's correct bit-flip errors first
(Just along with the Z axis)

Quantum bit-flip errors
are about the states.



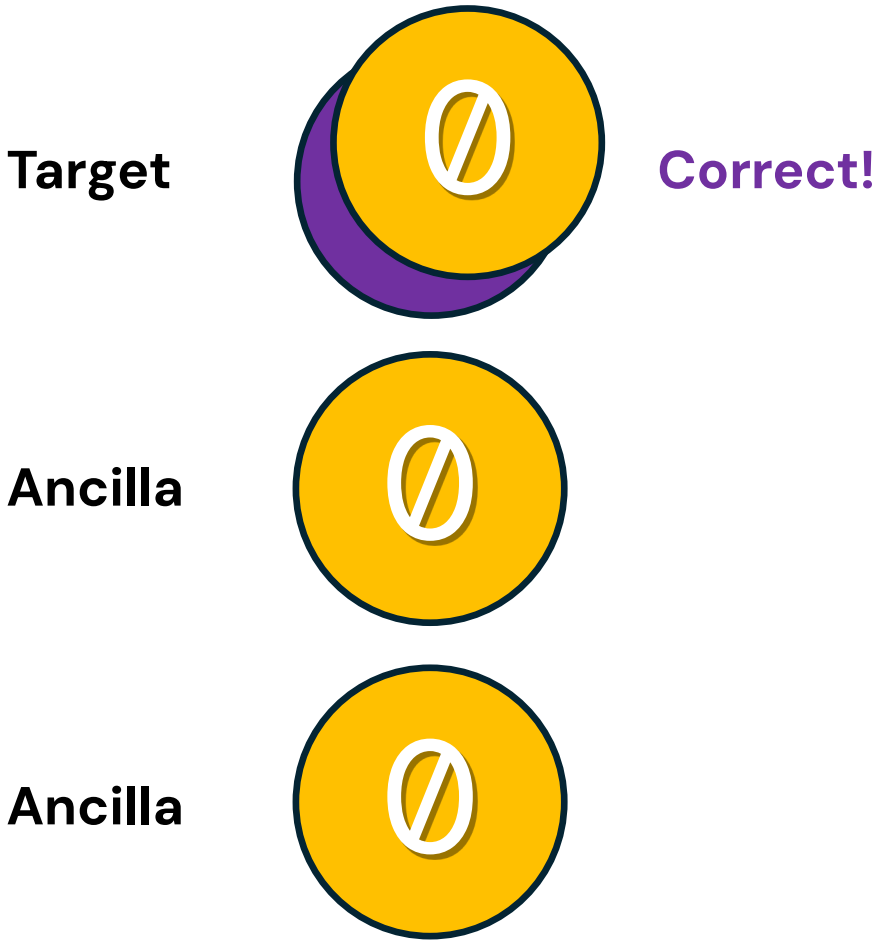
Quantum bit flip error correction

Let's correct bit-flip errors first
(Just along with the Z axis)



Quantum bit flip error correction

Let's correct bit-flip errors first
(Just along with the Z axis)



To do so, we can measure qubits at the end or in the middle.

Final measurement

Adjust each measurement outcome by looking at the ancilla qubits

Mid-circuit measurement

Measure qubits in the middle of the circuit and correct it as needed

More preferred because it can actively fix error in the middle of execution

But, don't qubits collapse after MCM?

They do.

So, we recover the qubits' phases and states
using quantum teleportation and feed-forward operations.
(a story for another day)

Quantum computers have another type of error: ***phase!***

Phase errors are also similarly fixed.

Qubits needed for bit-flip errors = 3

Qubits needed for phase-flip errors = 3

Total qubits needed for all errors = 3 × 3 = 9

The 9-qubit Shor code

Implying that fault-tolerant quantum computers need 9 times more physical qubits than the problem needs logically.

There are different methods for correcting errors.

Some representative methods

9-qubit Shor code

7-qubit Stean code

5-qubit code

Probably, more papers are being uploaded on arXiv at this moment.

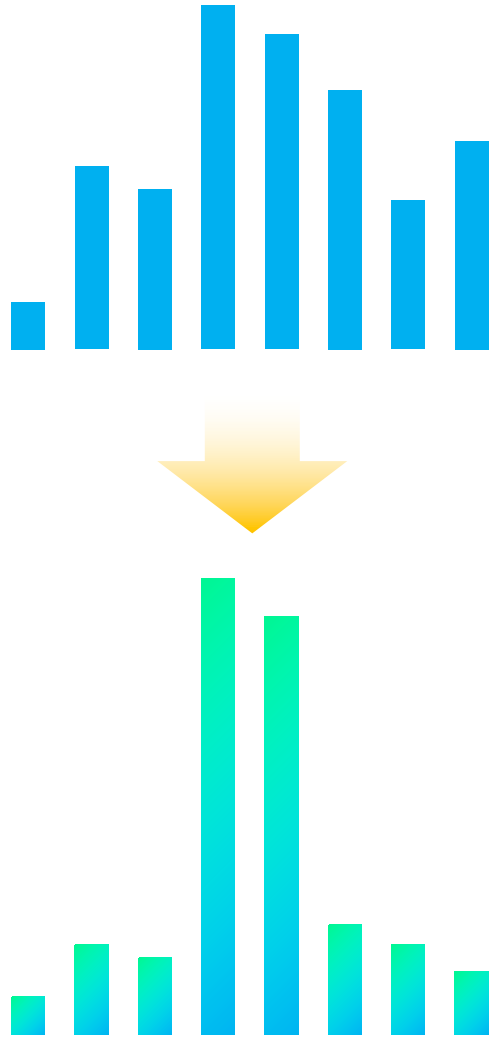
Quantum error mitigation is different than correction.

Error mitigation is about post-processing
by looking at the fidelity information of machine
and using some classical algorithmic methods.

It aims to reduce noise
in our measurement outcomes.

It does not need extra physical qubits.

We still need error correction though
because error mitigation is not so scalable.



Part 1

Introducing Quantum Computing

Topics covered

~~Quantum computers—qubit, gate, measure~~

~~Superposition, Bloch sphere, and entanglement~~

~~Shor's algorithm~~

~~Search algorithm (oracle-based)~~

~~Optimization problems~~

~~Quantum error correction (and mitigation)~~